



FIAP – Faculdade de Informática e Administração Paulista

DEFESA CIBERNÉTICA



DISCIPLINA: GLOBAL SOLUTION 2



RM566932



ALUNO: LEONARDO HENRIQUE BORGES DA SILVA

LEONARDO HENRIQUE BORGES DA SILVA
RM566932

RELATÓRIO GLOBAL SOLUTION - 2º SEMESTRE

Relatório do 2º Semestre do curso de Defesa Cibernética referente ao projeto Global Solution, contemplando os desafios (CTFs) disponibilizados para resolução, bem como a metodologia empregada, evidências coletadas e os resultados obtidos durante as atividades práticas.

Professor: Marcelo Gomes dos Santos

Coordenador: André Luis Cirino

RESUMO

LEONARDO HENRIQUE BORGES DA SILVA. Relatório Global Solution – 2º Semestre. Defesa Cibernética, 2026.

Este relatório apresenta a documentação técnica das atividades desenvolvidas durante o projeto Global Solution do curso de Defesa Cibernética. O trabalho consistiu na resolução de diversos desafios do tipo Capture The Flag (CTF), abordando diferentes áreas da Segurança da Informação, como análise forense digital, engenharia reversa, análise de tráfego de rede, segurança em redes sem fio, análise de firmware e identificação de informações ocultas em arquivos e protocolos de comunicação.

Durante a execução dos desafios, foram aplicados conceitos fundamentais de investigação digital e análise de sistemas, permitindo a identificação de vulnerabilidades, extração de informações ocultas e interpretação de dados codificados. As atividades exigiram raciocínio lógico, capacidade analítica e conhecimento técnico para compreender diferentes cenários simulados de segurança.

As principais tecnologias e ferramentas utilizadas incluíram sistemas operacionais Linux, Python, Wireshark, Tshark, Binwalk, John the Ripper, x64dbg e utilitários voltados para análise de arquivos, firmware e tráfego de rede. A utilização dessas ferramentas permitiu desenvolver habilidades práticas relacionadas à coleta de evidências, análise de artefatos digitais, engenharia reversa e resolução de problemas em ambientes de segurança ofensiva e defensiva.

Os desafios propostos contribuíram para o aprimoramento das competências técnicas necessárias para atuação na área de Segurança da Informação, proporcionando experiência prática em situações semelhantes às encontradas em investigações forenses, testes de invasão e análise de incidentes de segurança.

SUMÁRIO

RESUMO	3
1 INTRODUÇÃO	5
2 FERRAMENTAS UTILIZADAS	5
3 DESENVOLVIMENTO DOS CHALLENGES	7
3.1 A Carta do General	7
3.2 Analista Desaparecido	9
3.3 O Código do Cofre Virtual	11
3.4 Mistério em Meio aos Pacotes	14
3.5 Latência de Concessão DHCP	16
3.6 O Bilhete da Estação Abandonada	19
3.7 Terminal Legado	22
3.8 O Código dos Engenheiros	24
3.9 Firmware v5 – Reconstrução da Flag Fragmentada	27
3.10 Comunicação Ofuscada em Rede	32
3.11 Hardware Hacking EEPROM v7: Hex Oculto	35
3.12 Sinal Entre as Sombras	37
3.13 Pacotes Múltiplos	41
3.14 Shell Secreto	44
3.15 Hardware Hacking: Ruído Hexadecimal	48
4 CONCLUSÃO	52
5 REFERÊNCIAS	53

1 INTRODUÇÃO

Nesta atividade foi desenvolvido um conjunto de desafios do tipo Capture The Flag (CTF) com o objetivo de aplicar conhecimentos práticos relacionados à Segurança da Informação. Os desafios abordaram diferentes áreas da Defesa Cibernética, incluindo análise forense, engenharia reversa, análise de firmware, análise de tráfego de rede, criptografia e identificação de informações ocultas.

A proposta da atividade consistiu em simular situações encontradas em ambientes corporativos e investigações digitais, exigindo a utilização de ferramentas especializadas e técnicas de análise para identificação e extração de informações relevantes. Todas as etapas realizadas durante a resolução dos desafios foram documentadas neste relatório, apresentando os procedimentos executados, as evidências coletadas e os resultados obtidos.

2 FERRAMENTAS UTILIZADAS

Durante a execução dos desafios foram utilizadas ferramentas amplamente empregadas em atividades de análise forense, engenharia reversa e segurança ofensiva. A seguir são apresentadas as principais ferramentas utilizadas ao longo do projeto.

Kali Linux

Sistema operacional utilizado como ambiente principal para execução das análises, desenvolvimento de scripts e utilização das ferramentas de segurança.

Python 3

Linguagem de programação utilizada para automatizar tarefas de análise, decodificação de dados e desenvolvimento de scripts auxiliares para resolução dos desafios.

Wireshark

Ferramenta utilizada para análise de tráfego de rede, permitindo a inspeção detalhada de pacotes, protocolos e informações transmitidas durante as capturas analisadas.

Tshark

Versão em linha de comando do Wireshark utilizada para filtragem e extração de informações específicas presentes nos arquivos de captura de rede.

Binwalk

Ferramenta utilizada para identificação e extração de sistemas de arquivos, firmwares e dados embarcados presentes em imagens binárias.

John the Ripper

Ferramenta empregada para análise e quebra de hashes durante os desafios que envolveram recuperação de credenciais.

x64dbg

Ferramenta de engenharia reversa utilizada para análise dinâmica de executáveis, identificação de funções internas e localização de informações ocultas.

Strings

Utilitário utilizado para extração de cadeias de caracteres legíveis presentes em arquivos binários e dumps de memória.

StegSolve

Ferramenta utilizada para análise de imagens e identificação de informações ocultas através de técnicas de esteganografia.

As ferramentas apresentadas permitiram realizar análises em diferentes contextos de segurança da informação, contribuindo para a identificação de dados ocultos, reconstrução de informações e resolução dos desafios propostos.

3 DESENVOLVIMENTO DOS CHALLENGES

3.1 A Carta do General

Enunciado

Durante a guerra digital, um dos generais enviou uma carta cifrada. Intercepte-a

SVNC{cynl_trg_clguba}

O objetivo consistia em identificar o método de cifragem utilizado e recuperar a mensagem original.

Metodologia

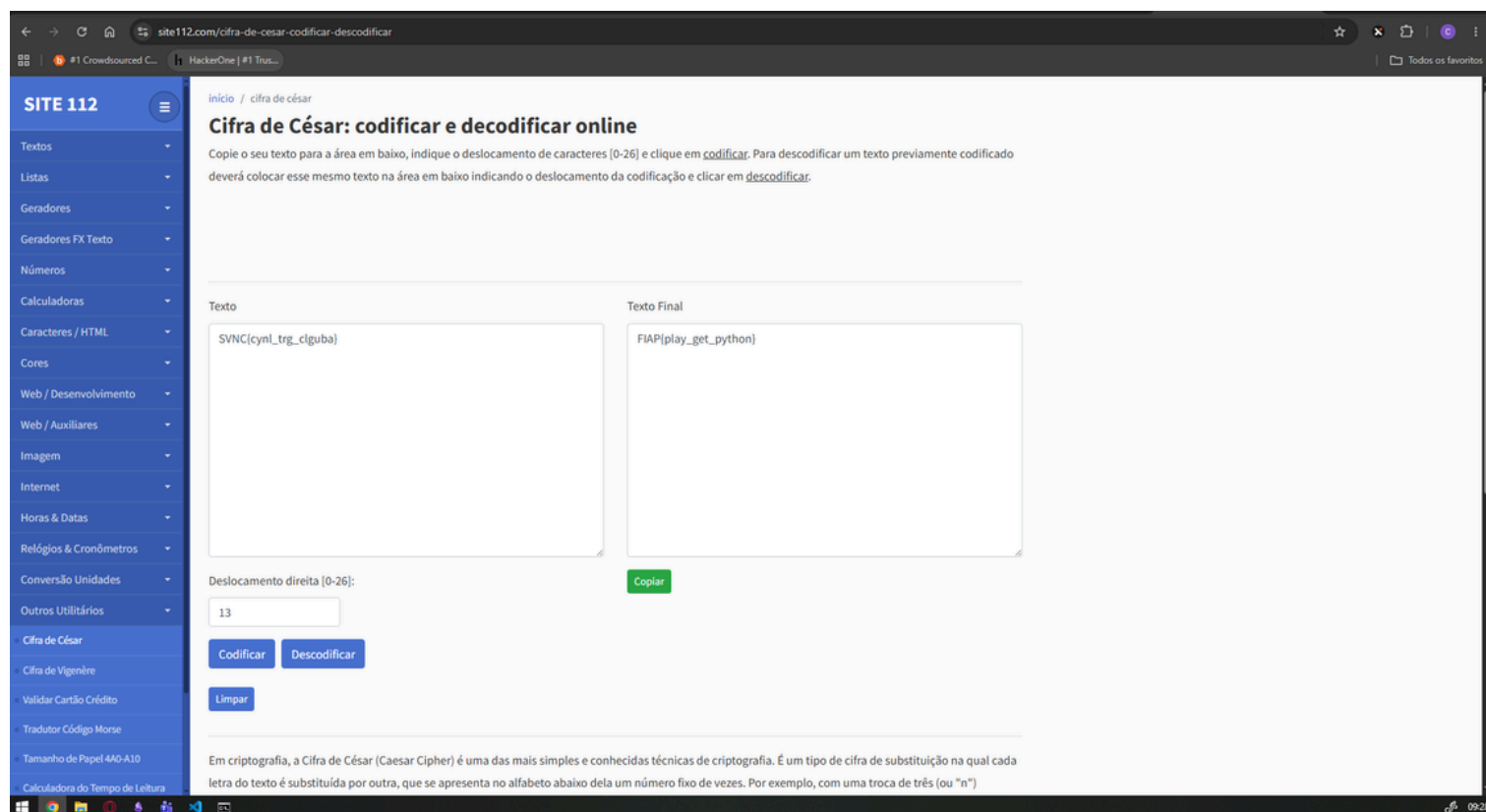
Inicialmente, foi analisado o conteúdo da mensagem e identificado que a estrutura apresentava características compatíveis com a cifra de César, especificamente a variante ROT13, amplamente utilizada em desafios de criptografia devido à sua simplicidade e reversibilidade.

Como primeira abordagem, foi utilizado o decodificador online disponível no endereço:

<https://site112.com/cifra-de-cesar-codificar-descodificar>

Foi configurado o deslocamento de 13 posições (ROT13) para realizar a decodificação da mensagem.

Figura 1 – Decodificação da mensagem utilizando ferramenta online

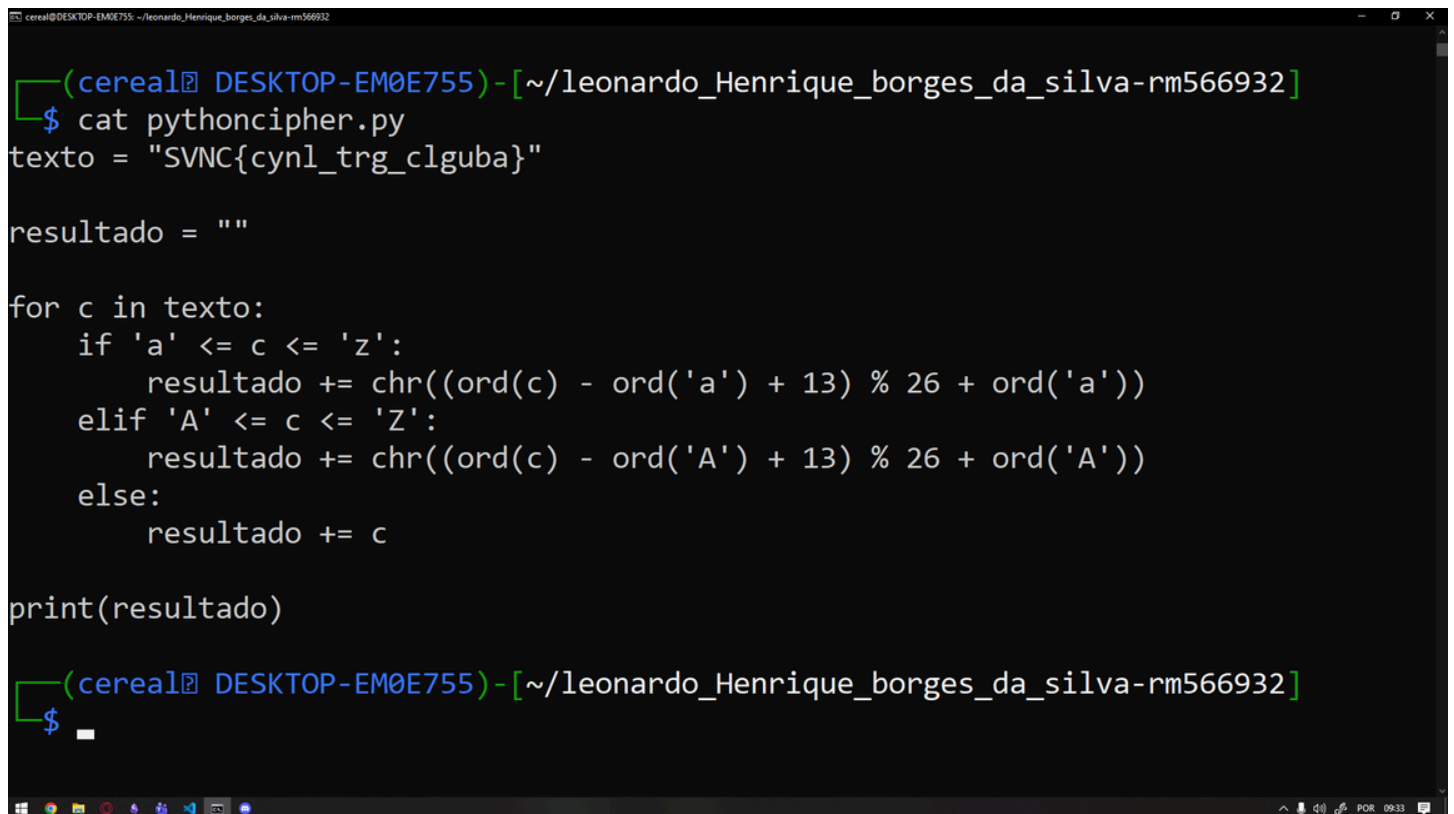


O resultado obtido foi:

FIAP{play_get_python}

Com o objetivo de validar o resultado obtido, foi desenvolvido um script em Python implementando a lógica da cifra de César com deslocamento de 13 posições.

Script Utilizado

A terminal window with a dark background and light-colored text. The window title is 'cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932'. The prompt is '(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]'. The user enters '\$ cat pythoncipher.py'. The script content is displayed: 'texto = "SVNC{cynl_trg_clguba}"', 'resultado = ""', a 'for' loop that iterates over each character 'c' in 'texto'. Inside the loop, it checks if 'c' is a lowercase letter ('a' to 'z') and shifts its ASCII value by 13, or if it's an uppercase letter ('A' to 'Z') and shifts its ASCII value by 13, or if it's not a letter, it leaves 'c' unchanged. The shifted character is added to 'resultado'. After the loop, 'print(resultado)' is executed. The prompt returns to '\$' with a cursor. The taskbar at the bottom shows various application icons and the system clock 'POR 09:33'.

```
(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat pythoncipher.py
texto = "SVNC{cynl_trg_clguba}"

resultado = ""

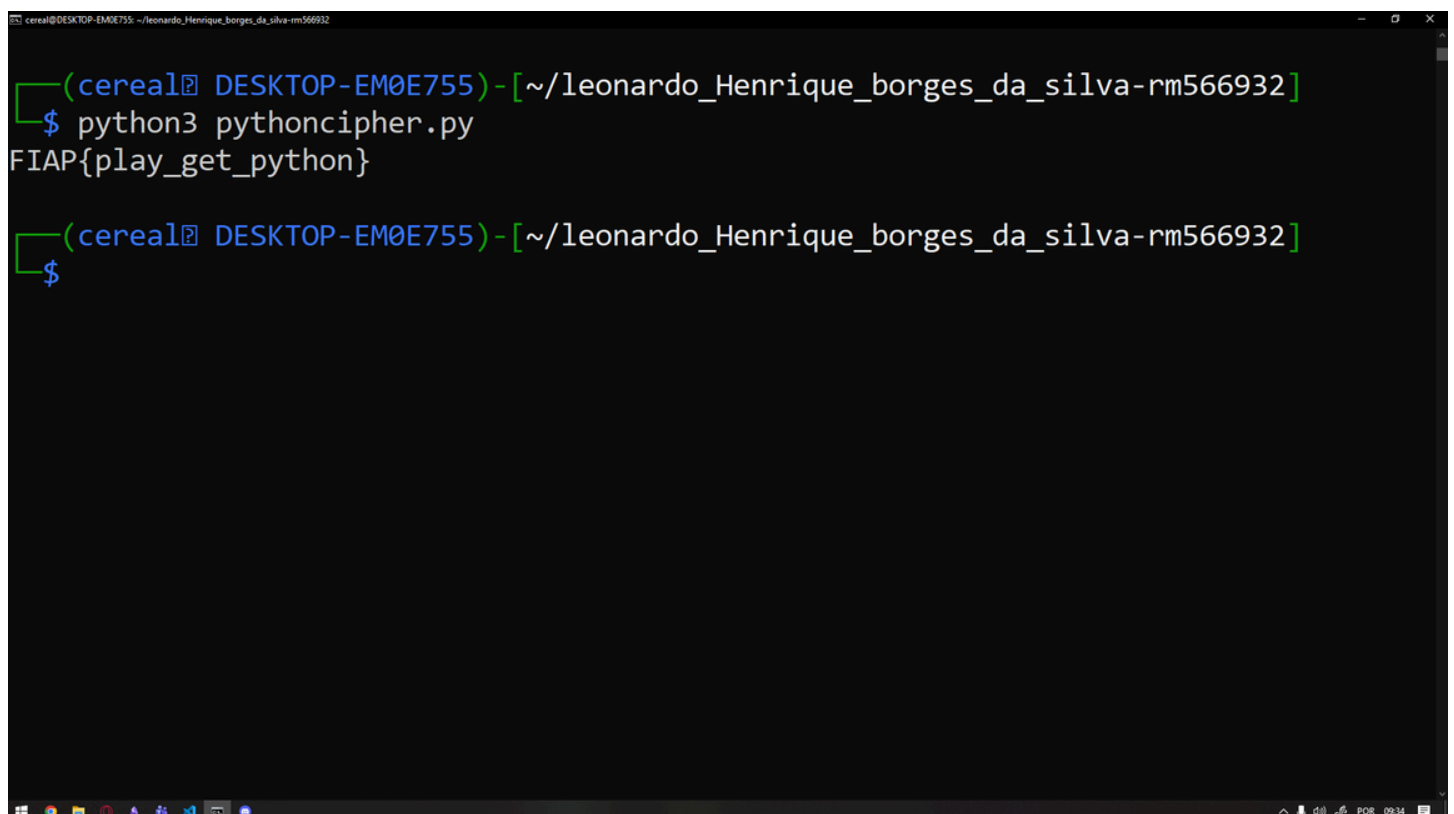
for c in texto:
    if 'a' <= c <= 'z':
        resultado += chr((ord(c) - ord('a') + 13) % 26 + ord('a'))
    elif 'A' <= c <= 'Z':
        resultado += chr((ord(c) - ord('A') + 13) % 26 + ord('A'))
    else:
        resultado += c

print(resultado)

(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

A execução do script produziu exatamente o mesmo resultado encontrado na ferramenta online.

Figura 2 – Execução do script Python para decodificação ROT13

A terminal window with a dark background and light-colored text. The window title is 'cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932'. The prompt is '(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]'. The user enters '\$ python3 pythoncipher.py'. The output 'FIAP{play_get_python}' is displayed. The prompt returns to '\$'. The taskbar at the bottom shows various application icons and the system clock 'POR 09:34'.

```
(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]
$ python3 pythoncipher.py
FIAP{play_get_python}

(cereal@ DESKTOP-EM0E755)-[~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Resultado

A análise confirmou que a mensagem havia sido cifrada utilizando o algoritmo ROT13. Após a decodificação, foi possível recuperar a flag do desafio:

FIAP{play_get_python}

A utilização de duas abordagens distintas permitiu validar a integridade do resultado obtido e confirmar o método de cifragem empregado.

3.2 Analista Desaparecido

Enunciado

Durante uma investigação interna, uma imagem foi recuperada do computador de um analista que desapareceu misteriosamente. A imagem parece comum à primeira vista, mas acredita-se que ele tenha deixado uma mensagem oculta essencial para a continuidade da operação.

Seu objetivo é descobrir o conteúdo oculto deixado por ele.

Arquivo: Flag.png

Metodologia

Inicialmente foi realizada uma análise preliminar do arquivo para identificar possíveis técnicas de ocultação de informações. Considerando que o desafio envolvia uma imagem PNG, levantou-se a hipótese de utilização de esteganografia.

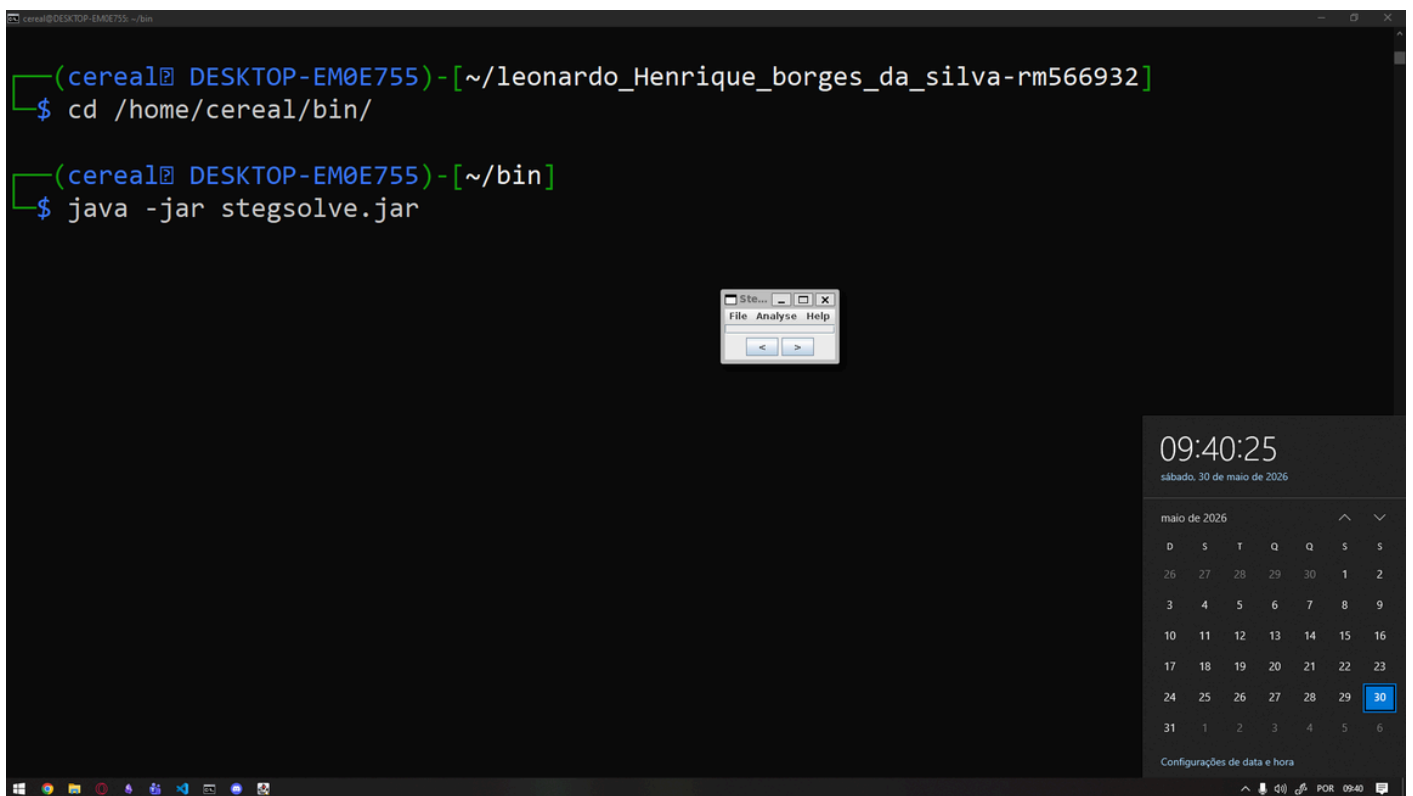
Para realizar a análise, foi utilizada a ferramenta StegSolve, amplamente empregada na identificação de conteúdos ocultos em imagens através da inspeção de canais de cor, planos de bits e camadas de pixels.

A ferramenta foi executada através do comando:

java -jar stegsolve.jar

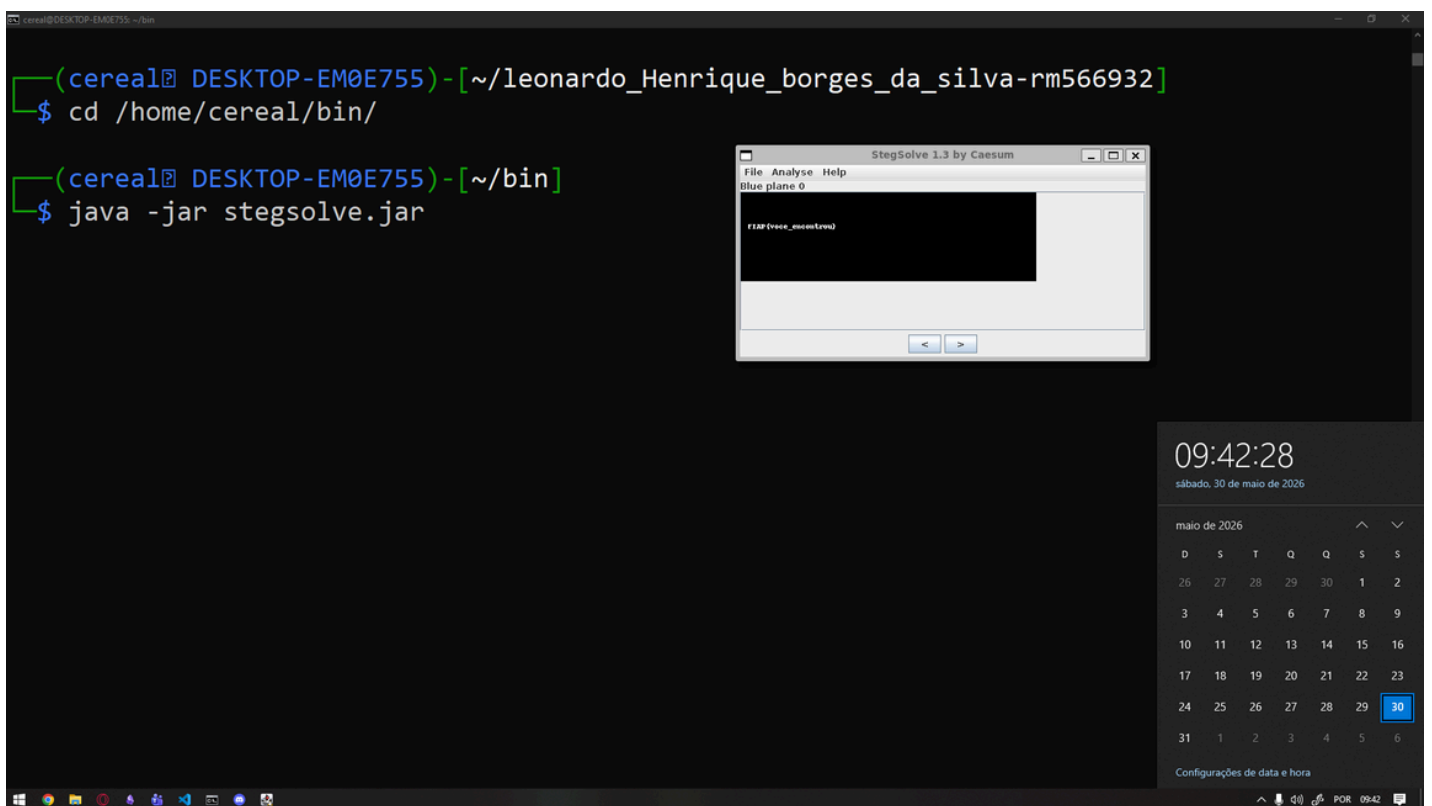
Após abrir a imagem na ferramenta, foram analisados os diferentes canais RGB e os planos de bits disponíveis, buscando identificar padrões visuais ou informações ocultas que não eram perceptíveis na visualização convencional da imagem.

Figura 3 – Inicialização da ferramenta StegSolve



Durante a navegação pelos planos de bits e canais de cor, foi possível identificar a presença de informações ocultas incorporadas à imagem.

Figura 4 – Análise dos canais da imagem



Resultado

A análise esteganográfica permitiu recuperar a mensagem escondida pelo analista, possibilitando a obtenção da flag solicitada pelo desafio.

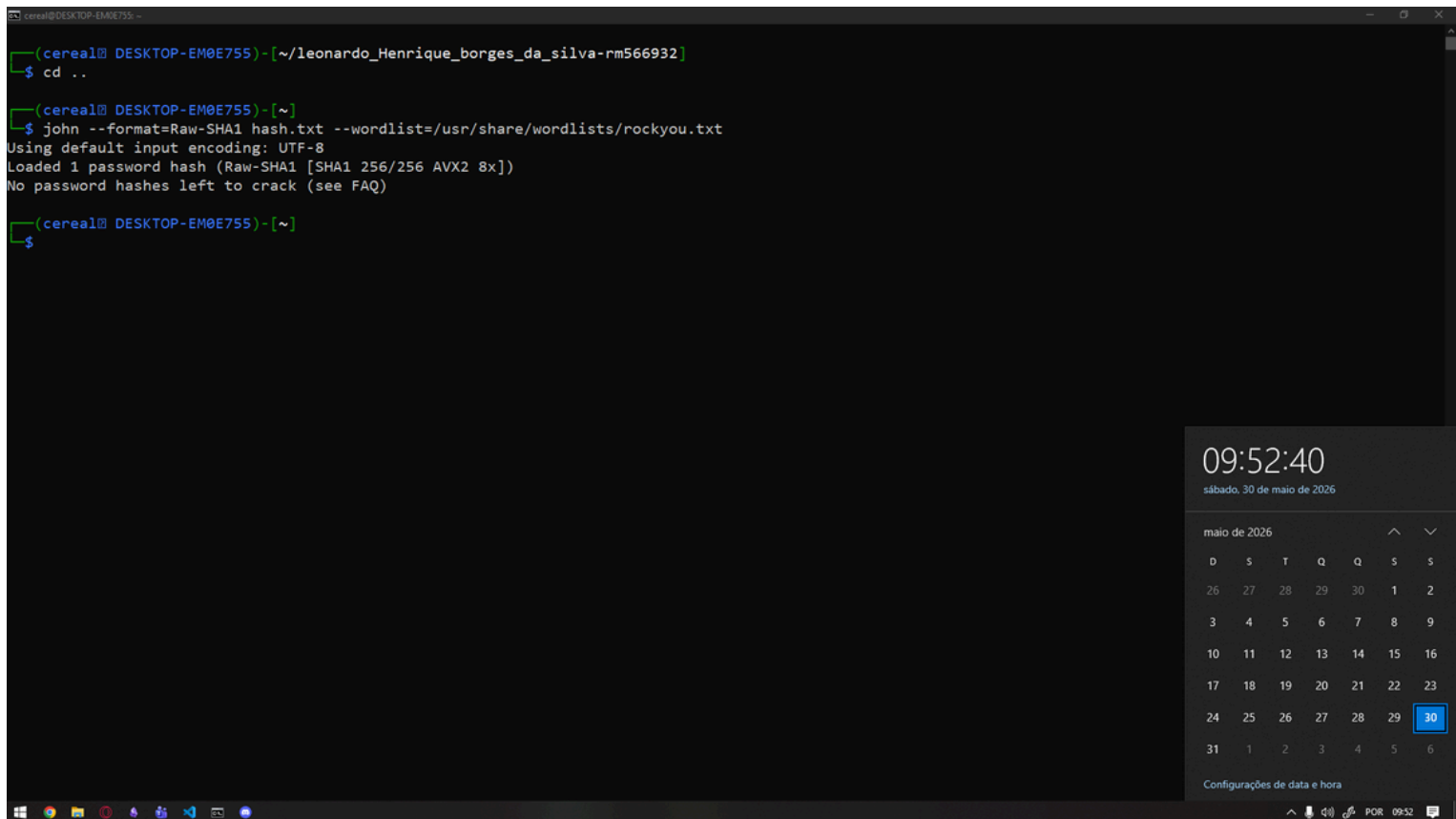
Flag obtida: FIAP{voce_encontrou}

Após a preparação da wordlist, foi criado um arquivo contendo o hash fornecido pelo desafio.

Em seguida, foi executado o ataque de dicionário utilizando o John the Ripper:

```
john --format=Raw-SHA1 hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

Figura 6 – Execução do John the Ripper para quebra do hash SHA-1



Após alguns instantes, a ferramenta conseguiu identificar a senha correspondente ao hash.

Para visualizar novamente o resultado armazenado pelo John, foi utilizado o comando:

```
john --show hash.txt
```

Figura 7 – Exibição da senha recuperada


```
cereal@DESKTOP-EM0E755: ~  
└─$ cd ..  
  
└─$ john --format=Raw-SHA1 hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])  
No password hashes left to crack (see FAQ)  
  
└─$ john --show hash.txt  
?:Passw0rd!  
  
1 password hash cracked, 0 left  
  
└─$
```

Resultado

Através da utilização do John the Ripper e da wordlist RockYou foi possível recuperar a senha correspondente ao hash SHA-1 fornecido no desafio.

Com a senha identificada, foi possível montar a flag no formato solicitado pelo enunciado e concluir a resolução do challenge.

Flag obtida:

FIAP{Passw0rd!}

3.4 Mistério em Meio aos Pacotes

Enunciado

Durante a investigação de um incidente em uma rede corporativa, foi capturado um trecho de tráfego contendo exatamente 100 pacotes TCP direcionados à porta 443.

Apesar da aparência legítima, uma análise mais minuciosa pode revelar um conteúdo incomum transmitido de forma discreta por um dos dispositivos conectados.

Seu objetivo é identificar a mensagem transmitida de maneira camuflada por um dos pacotes nesta sessão.

Arquivo: pacotes.pcap

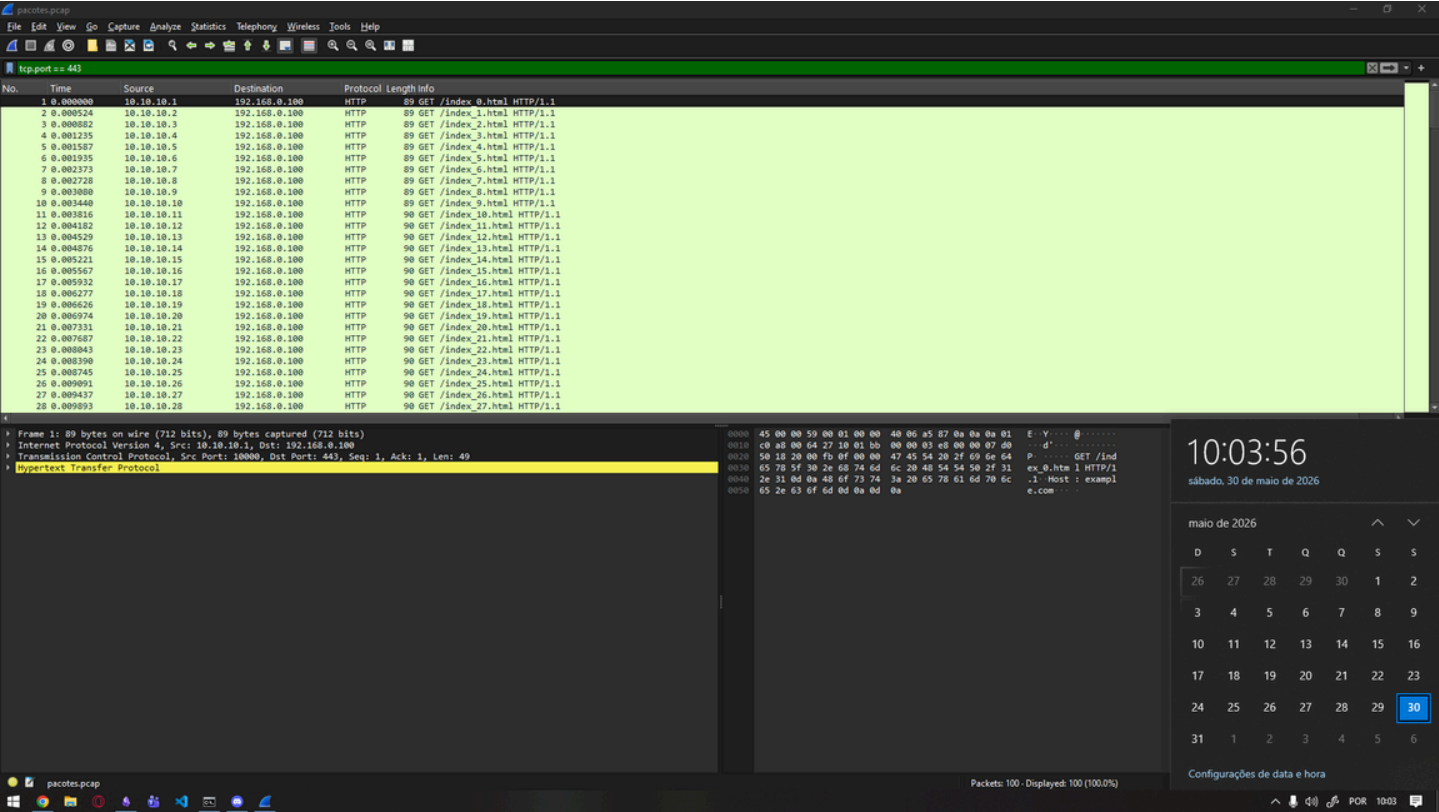
Metodologia

Inicialmente foi aberto o arquivo de captura utilizando o Wireshark para análise dos pacotes presentes.

Considerando a informação fornecida pelo enunciado, foi aplicado um filtro para exibir apenas os pacotes TCP destinados à porta 443.

tcp.port == 443

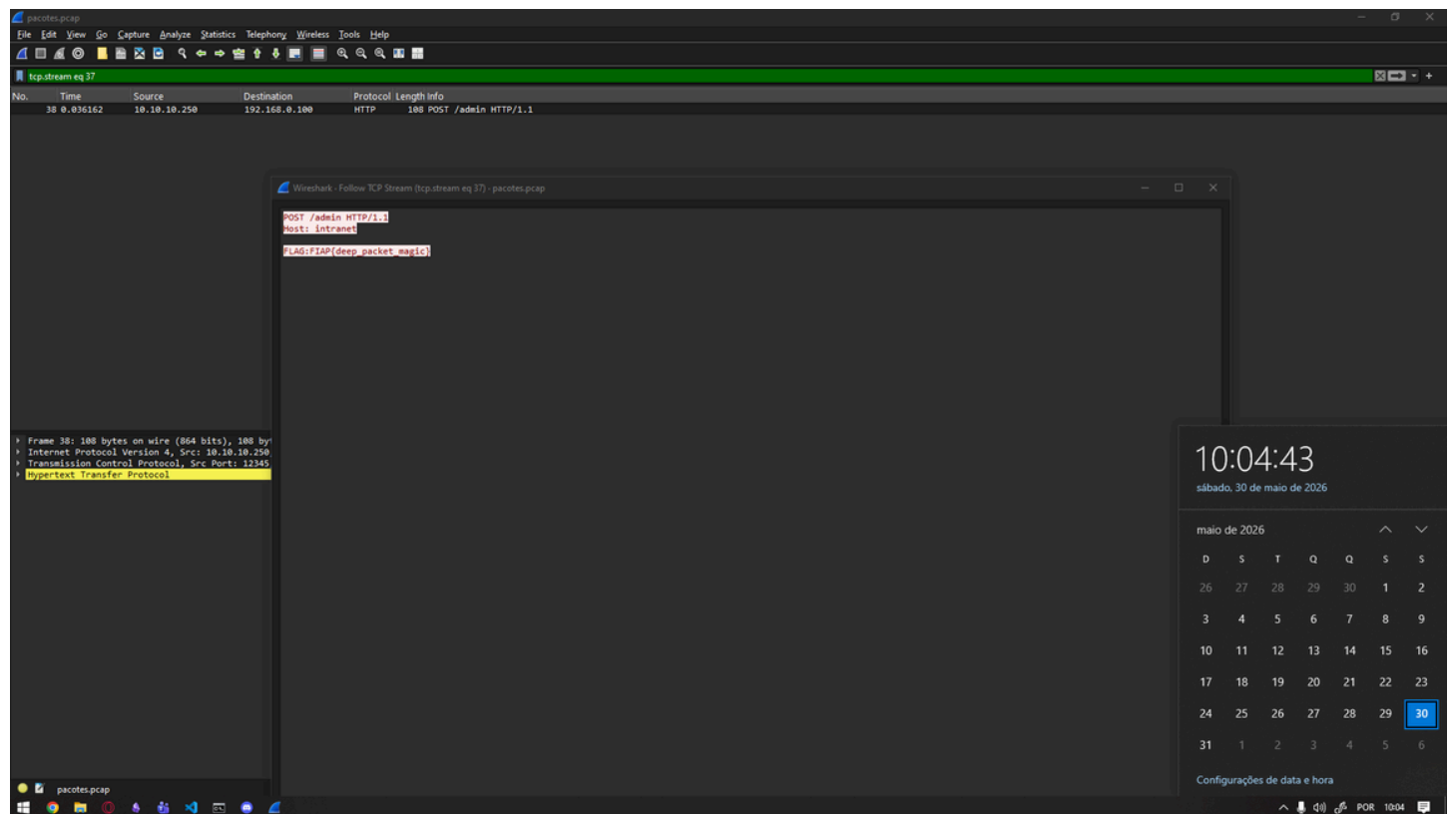
Figura 7 - Aplicação do filtro para análise dos pacotes TCP na porta 443



Após a filtragem dos pacotes, foi realizada uma inspeção detalhada dos campos de cada pacote em busca de padrões incomuns, dados codificados ou informações inseridas em campos específicos do protocolo.

Durante a análise foi identificada uma mensagem escondida em um dos pacotes da captura.

Figura 8 – Identificação da mensagem oculta no pacote analisado



A mensagem encontrada foi extraída e utilizada para compor a flag exigida pelo desafio.

Resultado

A análise dos pacotes permitiu localizar a informação transmitida de forma camuflada dentro da sessão de rede.

A mensagem recuperada foi utilizada para obtenção da flag correspondente ao challenge.

Flag obtida:

FIAP{deep_packet_magic}

3.5 Latência de Concessão DHCP

Enunciado

Durante uma simulação de rede, pacotes de negociação DHCP foram capturados para análise de desempenho. Em meio a pacotes UDP irrelevantes, há uma requisição DHCP Discover legítima e, posteriormente, uma DHCP Offer da rede.

Seu objetivo é identificar o tempo decorrido entre os dois eventos relacionados à concessão de endereço.

Formato da resposta: FIAP{segundos.decimal}

Qual foi o tempo decorrido entre o envio do pacote DHCP Discover e o recebimento da resposta DHCP Offer?

Arredonde com precisão de duas casas decimais, ex: FIAP{0.00}

Arquivo: pacotes_dhcp.pcap

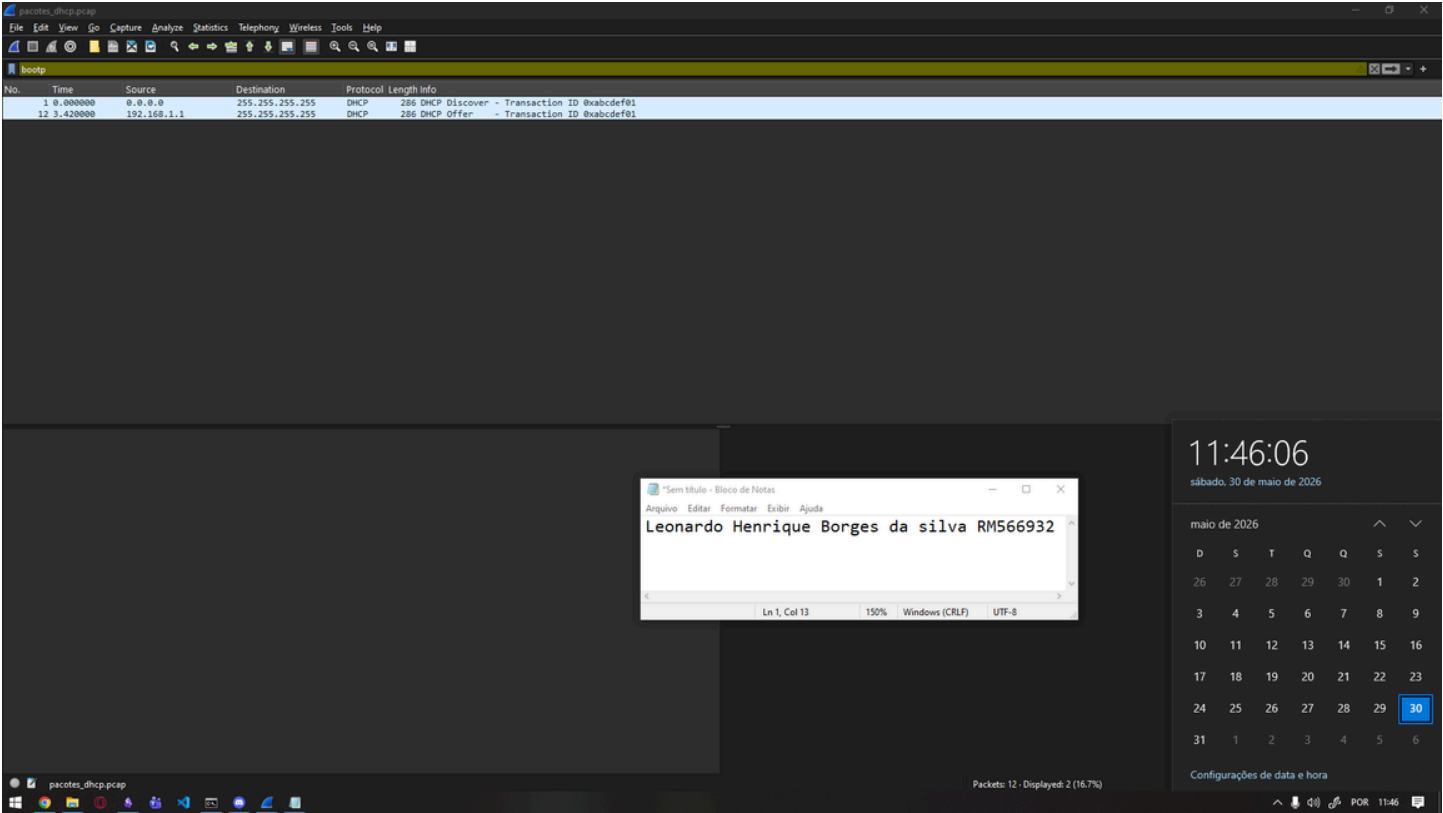
Metodologia

Inicialmente o arquivo de captura foi aberto utilizando o Wireshark para análise dos pacotes presentes.

Considerando que o desafio envolvia o protocolo DHCP, foi aplicado o seguinte filtro para exibir apenas os pacotes relacionados ao processo de obtenção de endereço IP:

bootp

Figura 9 – Aplicação do filtro BOOTP/DHCP no Wireshark



Após a filtragem, foram identificados os pacotes responsáveis pela negociação DHCP. Em seguida, foi localizado o pacote DHCP Discover, enviado pelo cliente para solicitar um endereço IP à rede.

Posteriormente, foi identificado o pacote DHCP Offer, enviado pelo servidor em resposta à solicitação realizada.

Figura 10 - Identificação do pacote DHCP Discover

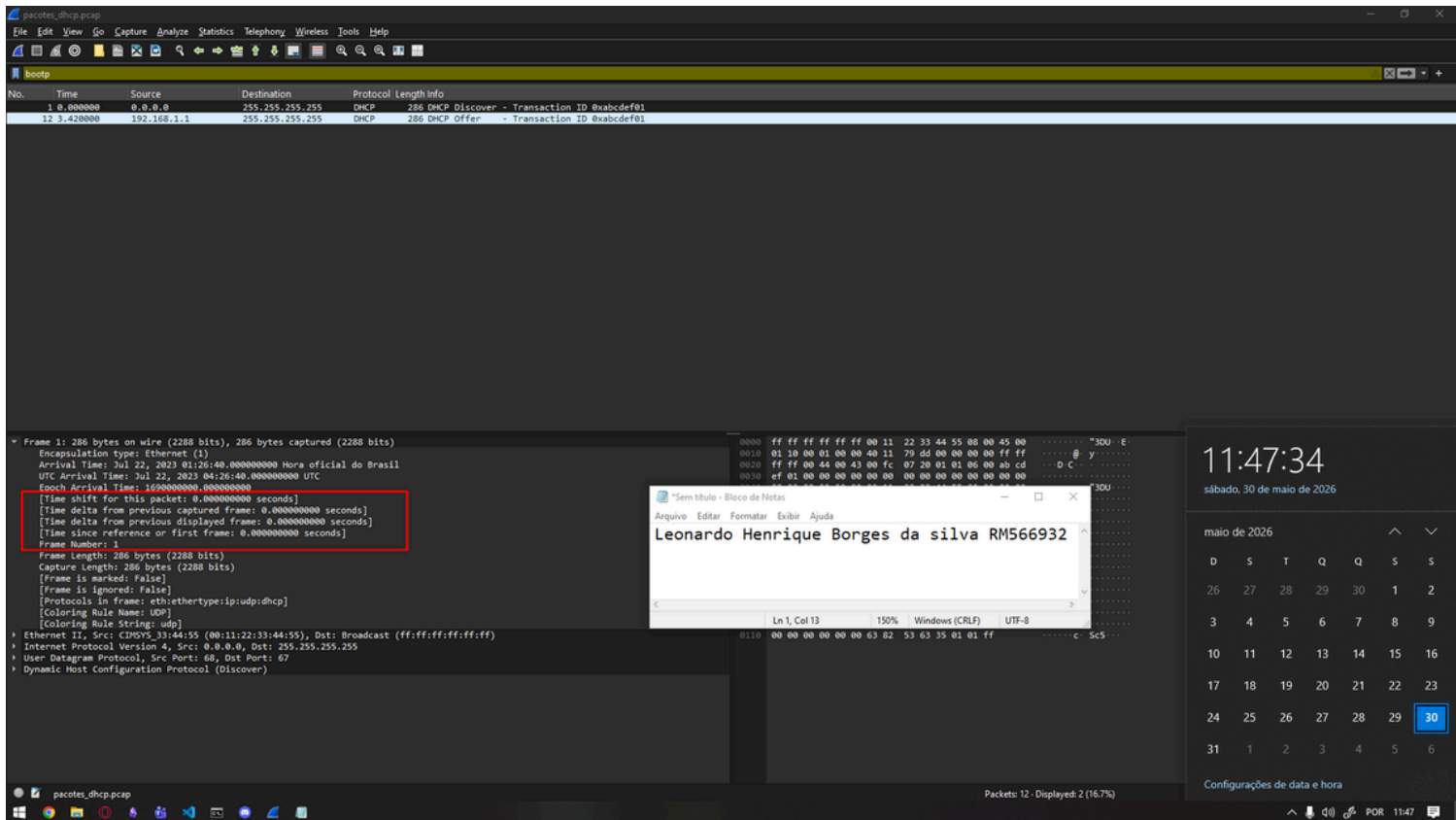
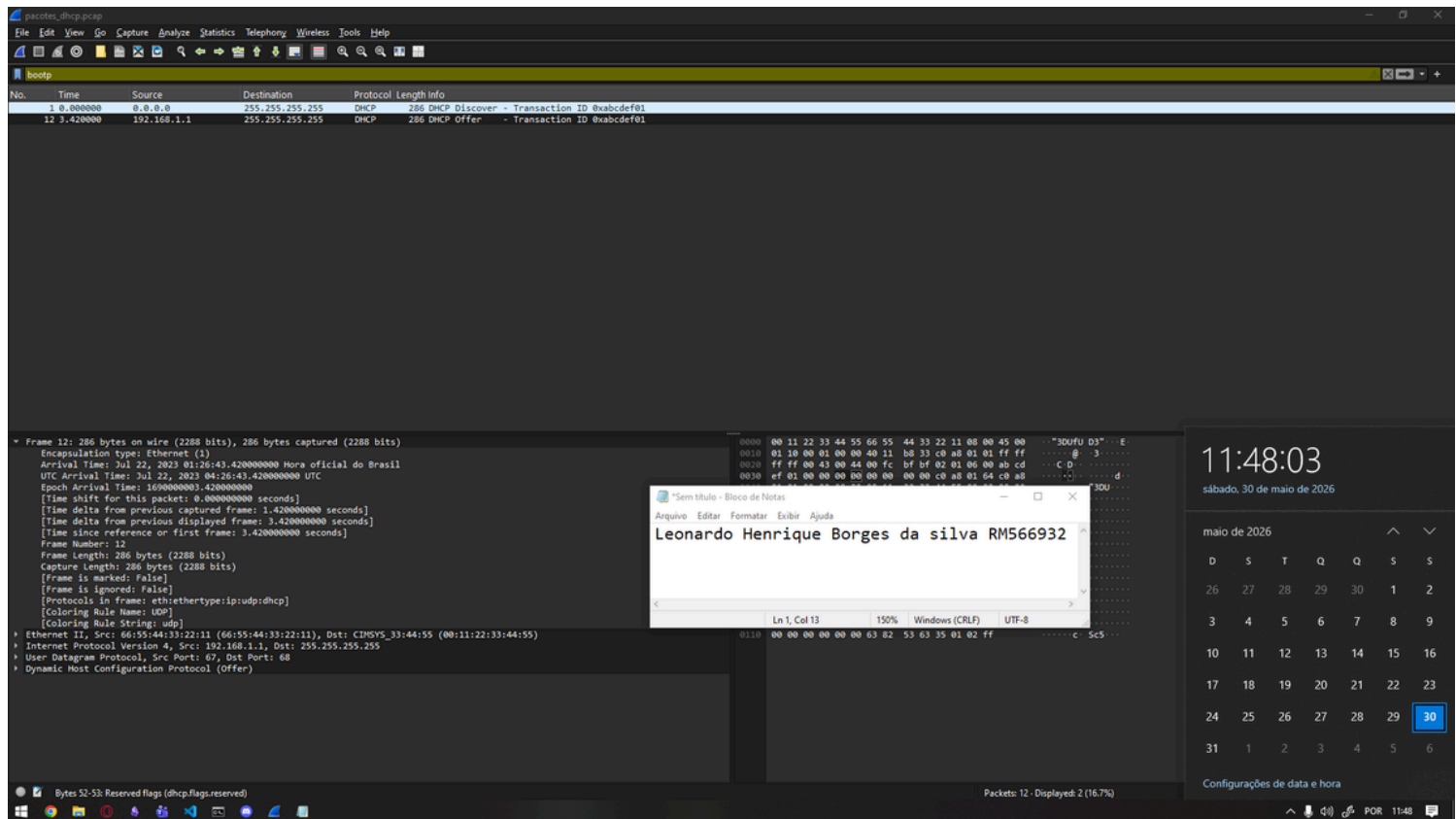


Figura 11 - Identificação do pacote DHCP Offer



Com ambos os pacotes identificados, foi realizada a comparação dos timestamps exibidos pelo Wireshark. A diferença entre os horários permitiu determinar a latência da concessão DHCP.

Para facilitar a visualização, também foi utilizada a funcionalidade de exibição de tempo relativo do Wireshark, permitindo medir com precisão o intervalo entre os eventos.

Resultado

A análise dos timestamps dos pacotes DHCP Discover e DHCP Offer permitiu calcular o tempo necessário para que o servidor respondesse à solicitação do cliente.

O valor encontrado foi utilizado para compor a flag solicitada pelo desafio.

Flag obtida:

FIAP{3.42}

Onde 3.42 representa o tempo calculado entre o envio da requisição DHCP Discover e o recebimento da resposta DHCP Offer, arredondado para duas casas decimais.

A atividade demonstrou a utilização do Wireshark para análise de protocolos de rede e medição de tempos de resposta em processos de atribuição dinâmica de endereços IP.

3.6 O Bilhete da Estação Abandonada

Enunciado

Durante uma simulação de reconhecimento em campo, foi encontrado um bilhete rasgado próximo a uma estação ferroviária abandonada. O texto aparenta ser ilegível à primeira vista, mas analistas acreditam que o autor utilizava um método rotacional simples para ocultar mensagens.

Texto encontrado:

fliudghfhvdu

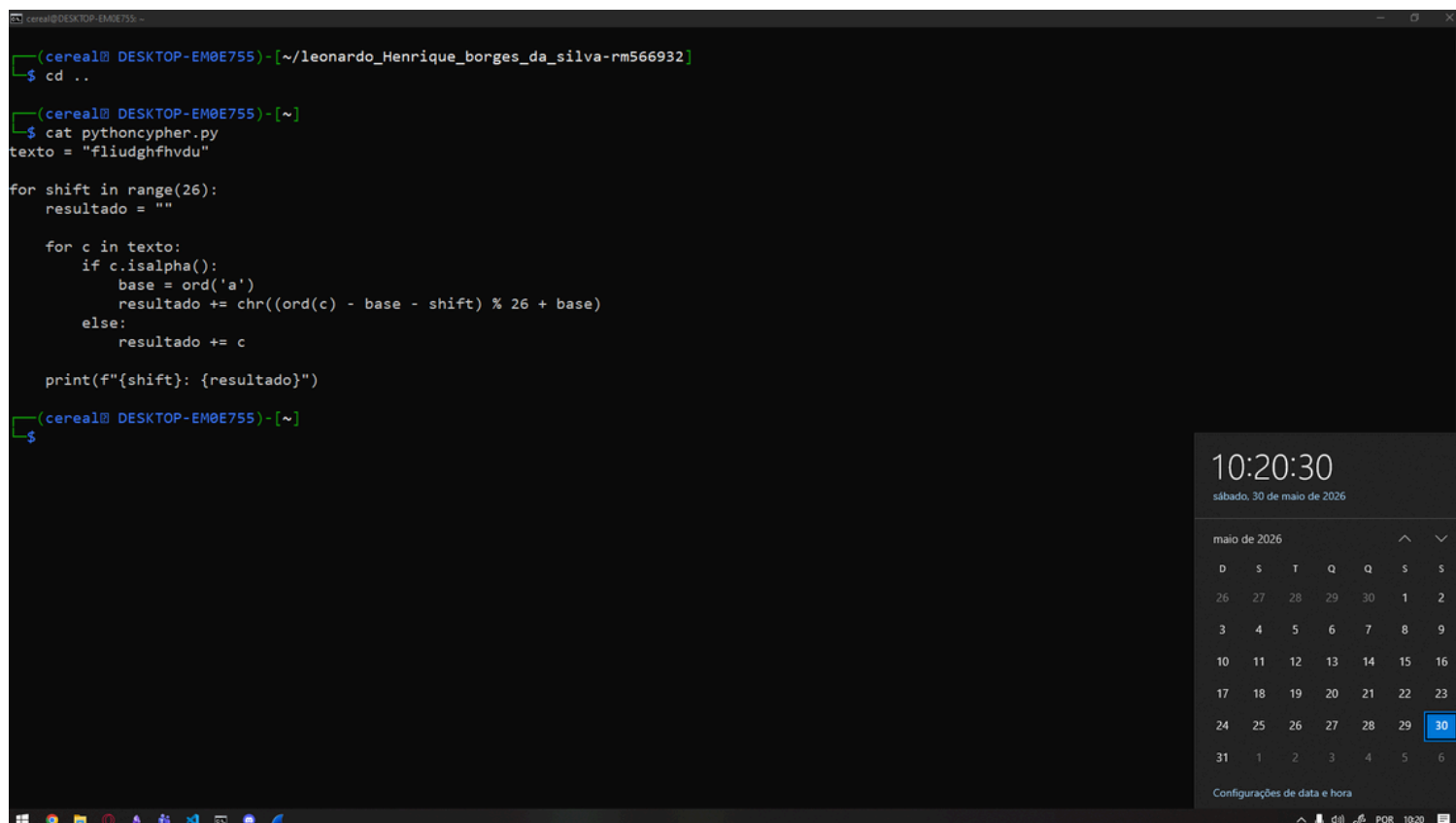
Seu objetivo é decifrar a mensagem original contida nesse bilhete.

Metodologia

Inicialmente foi realizada uma análise da string fornecida. Considerando a dica presente no enunciado sobre um método rotacional simples, levantou-se a hipótese de utilização da cifra de César, técnica criptográfica clássica baseada no deslocamento das letras do alfabeto.

Para validar a hipótese, foi desenvolvido um script em Python capaz de testar todos os 26 deslocamentos possíveis da cifra de César.

Script Utilizado



```
(cereal@DESKTOP-EM0E755) - [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cd ..

(cereal@DESKTOP-EM0E755) - [~]
$ cat pythoncypher.py
texto = "fliudghfhvdu"

for shift in range(26):
    resultado = ""

    for c in texto:
        if c.isalpha():
            base = ord('a')
            resultado += chr((ord(c) - base - shift) % 26 + base)
        else:
            resultado += c

    print(f"{shift}: {resultado}")

(cereal@DESKTOP-EM0E755) - [~]
$
```

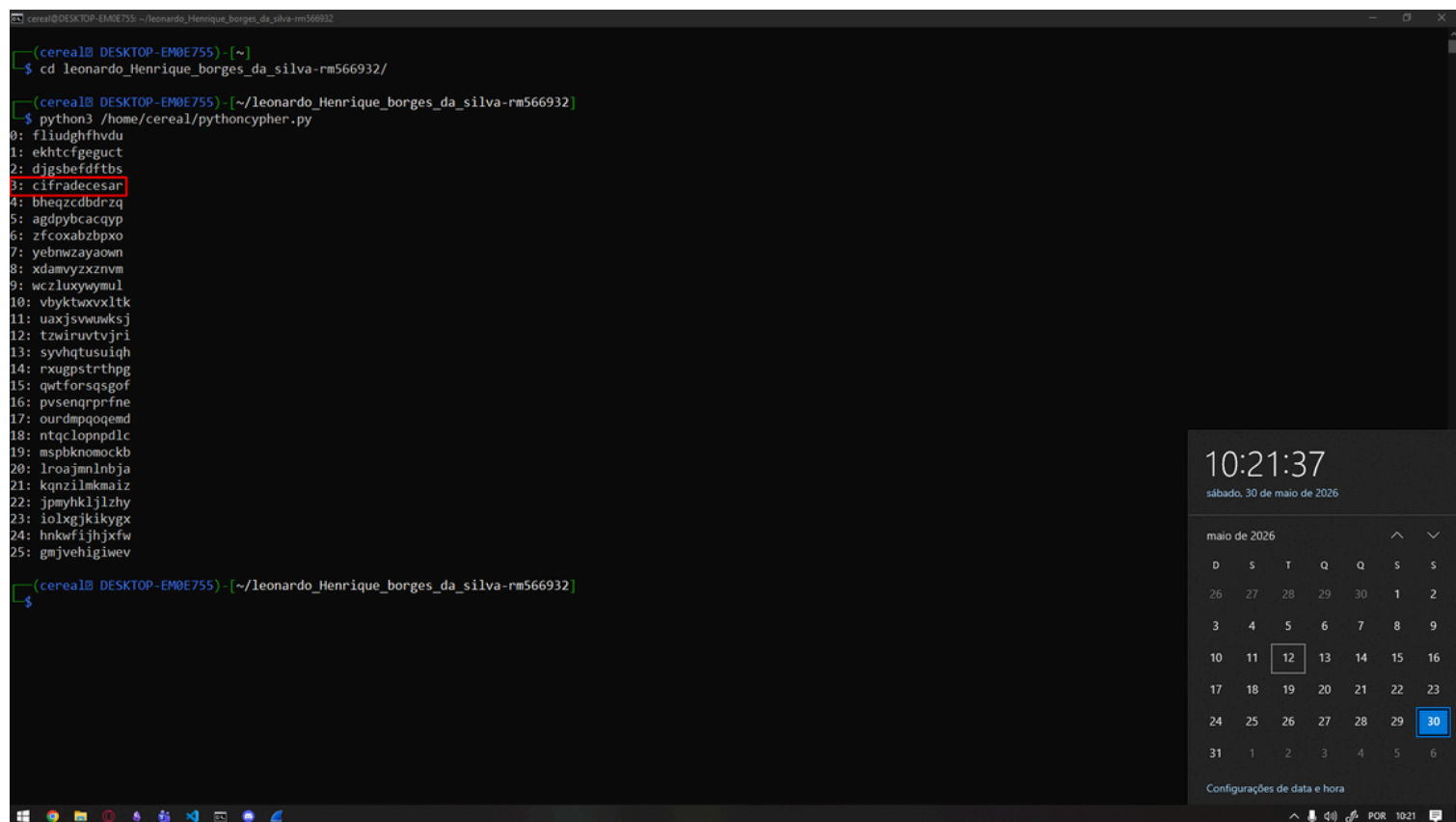
10:20:30
sábado, 30 de maio de 2026

maio de 2026

D	S	T	Q	Q	S	S
26	27	28	29	30	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Configurações de data e hora

Figura 12 – Execução do script para identificação do deslocamento da cifra de César



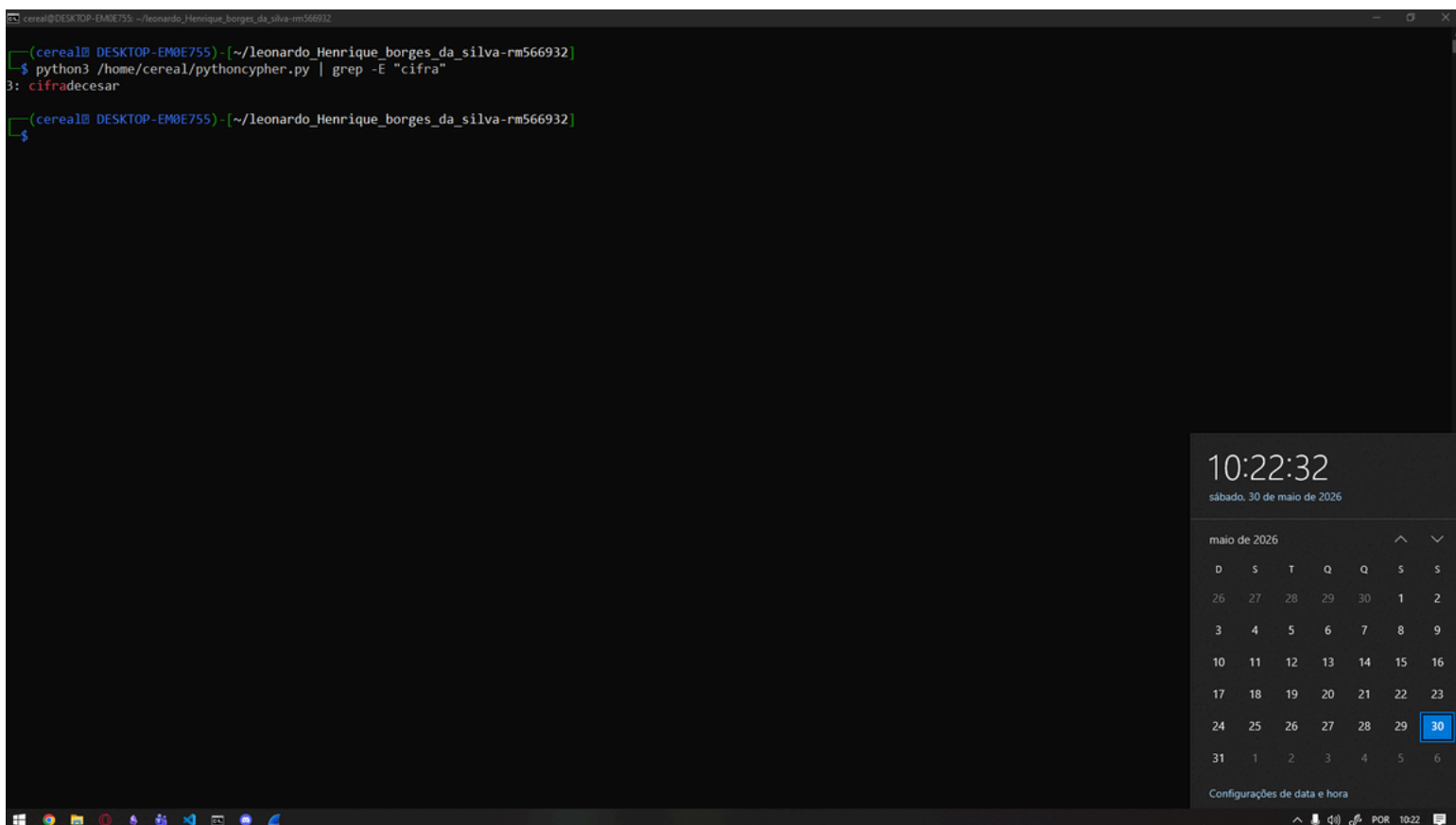
Após a execução do script, foi possível visualizar todos os possíveis resultados gerados pelos deslocamentos. Entre eles, um dos resultados produziu uma frase legível em inglês, indicando o deslocamento correto utilizado na cifragem.

A análise confirmou a utilização de um deslocamento de três posições, característico da cifra de César tradicional.

Figura 13 - Resultado da decodificação da mensagem


```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
(cereal@ DESKTOP-EM0E755) - [~/leonardo_Henrique_borges_da_silva-rm566932]
$ python3 /home/cereal/pythoncypher.py | grep -E "cifra"
3: cifradecesar

(cereal@ DESKTOP-EM0E755) - [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

The image shows a Windows desktop environment. A terminal window is open, displaying the execution of a Python script that filters for the word 'cifra', resulting in the output 'cifradecesar'. In the bottom right corner, a system clock and calendar overlay are visible, showing the time as 10:22:32 on Saturday, May 30, 2026. The taskbar at the bottom contains several application icons.

Resultado

Após a aplicação do deslocamento correto, a mensagem original foi recuperada com sucesso.

A partir da mensagem decodificada foi possível identificar a flag solicitada pelo desafio e concluir sua resolução.

Flag obtida:

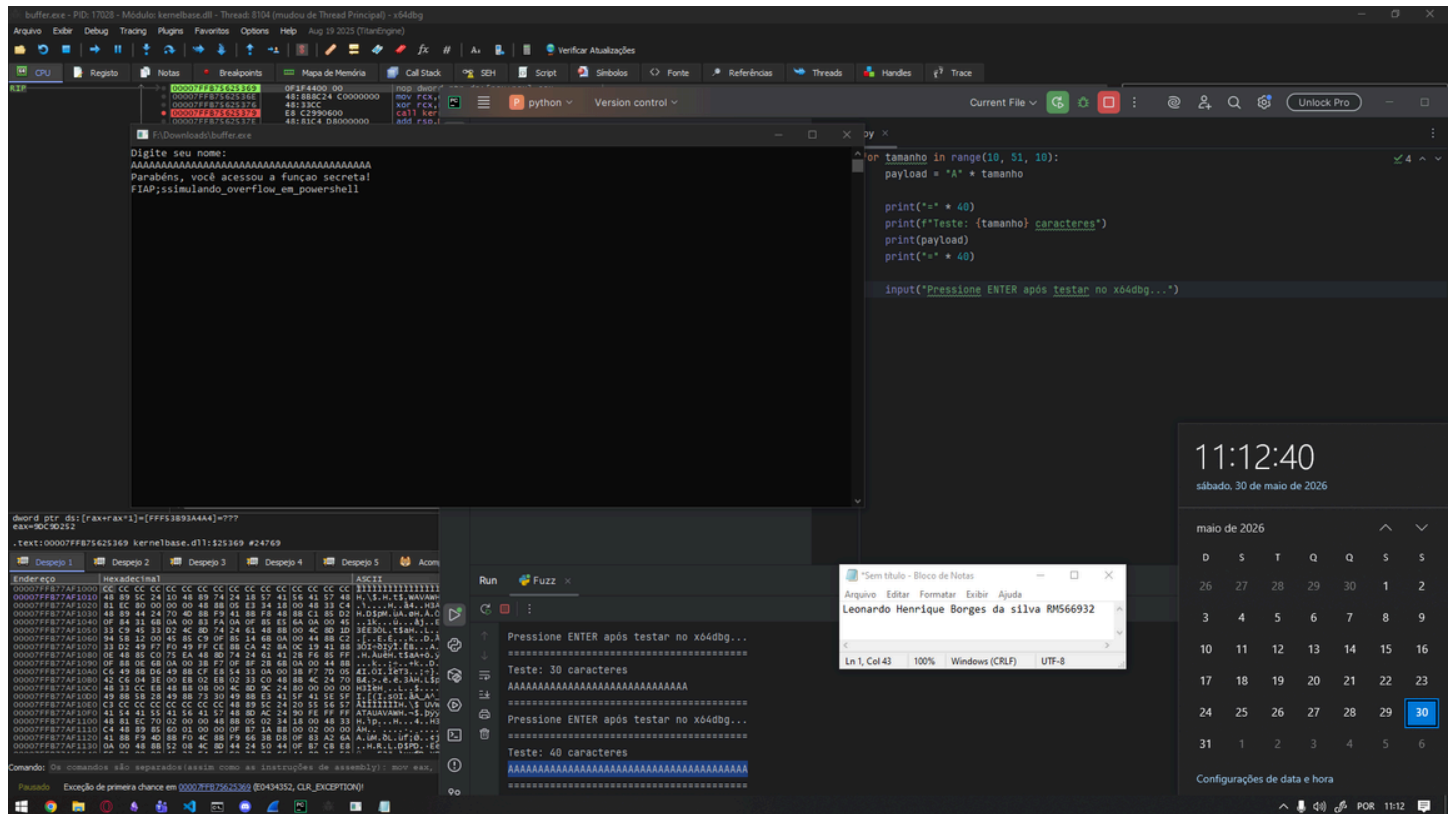
FIAP{cifradecesar}

Este desafio demonstrou a aplicação prática de conceitos básicos de criptografia clássica e a utilização de scripts para automatização do processo de análise e decodificação de mensagens cifradas.

Inicialmente foram realizados testes com 10, 20 e 30 caracteres, sem que fossem observadas alterações significativas no comportamento do programa.

Entretanto, ao fornecer uma entrada contendo 40 caracteres, foi identificado um comportamento inesperado: a aplicação deixava de executar seu fluxo normal e exibia uma mensagem secreta armazenada internamente.

Figura 15 – Execução da aplicação utilizando entrada com 40 caracteres



Esse comportamento indicou a existência de uma falha relacionada ao tratamento do buffer de entrada, permitindo o acesso à funcionalidade oculta prevista pelo desafio.

Resultado

Através da identificação do tamanho crítico do buffer e da manipulação controlada da entrada fornecida ao programa, foi possível acionar o comportamento oculto da aplicação e recuperar a mensagem secreta utilizada para compor a flag do desafio.

O valor obtido permitiu concluir a resolução da atividade com sucesso.

Flag obtida:

FIAP{simulando_overflow_em_powershell}

3.8 O Código dos Engenheiros

Enunciado

Uma equipe de engenharia embutiu um código de acesso especial em uma partição do firmware para recuperação de emergência. O código está presente em um dos arquivos da partição raiz e pode ser identificado por quem souber onde procurar.

Seu trabalho é extrair a flag embutida no sistema de arquivos do firmware.

Arquivo: firmware_router_v2.bin

Metodologia

Inicialmente foi realizada a análise do firmware utilizando a ferramenta Binwalk, amplamente utilizada para identificação e extração de sistemas de arquivos embarcados.

Foi executado o seguinte comando:

```
binwalk firmware_router_v2.bin
```

Figura 16 – Análise inicial do firmware utilizando Binwalk

```
cereal@DESKTOP-EM0E755: ~/leonardo_henrique_borges_da_silva-rm566932
$ binwalk /home/cereal/firmware_router_v2.bin
```

DECIMAL	HEXADECEIMAL	DESCRIPTION
524288	0x80000	gzip compressed data, maximum compression, has original file name: "rootfs_simulado.tar", last modified: 2025-05-15 22:04:42

```
cereal@DESKTOP-EM0E755: ~/leonardo_henrique_borges_da_silva-rm566932
$
```

The screenshot also shows a system clock overlay in the bottom right corner indicating the time is 11:16:40 on Saturday, 30 de maio de 2026, and a calendar for May 2026 with the 30th highlighted.

A análise revelou a presença de um arquivo compactado do tipo TAR incorporado ao firmware.

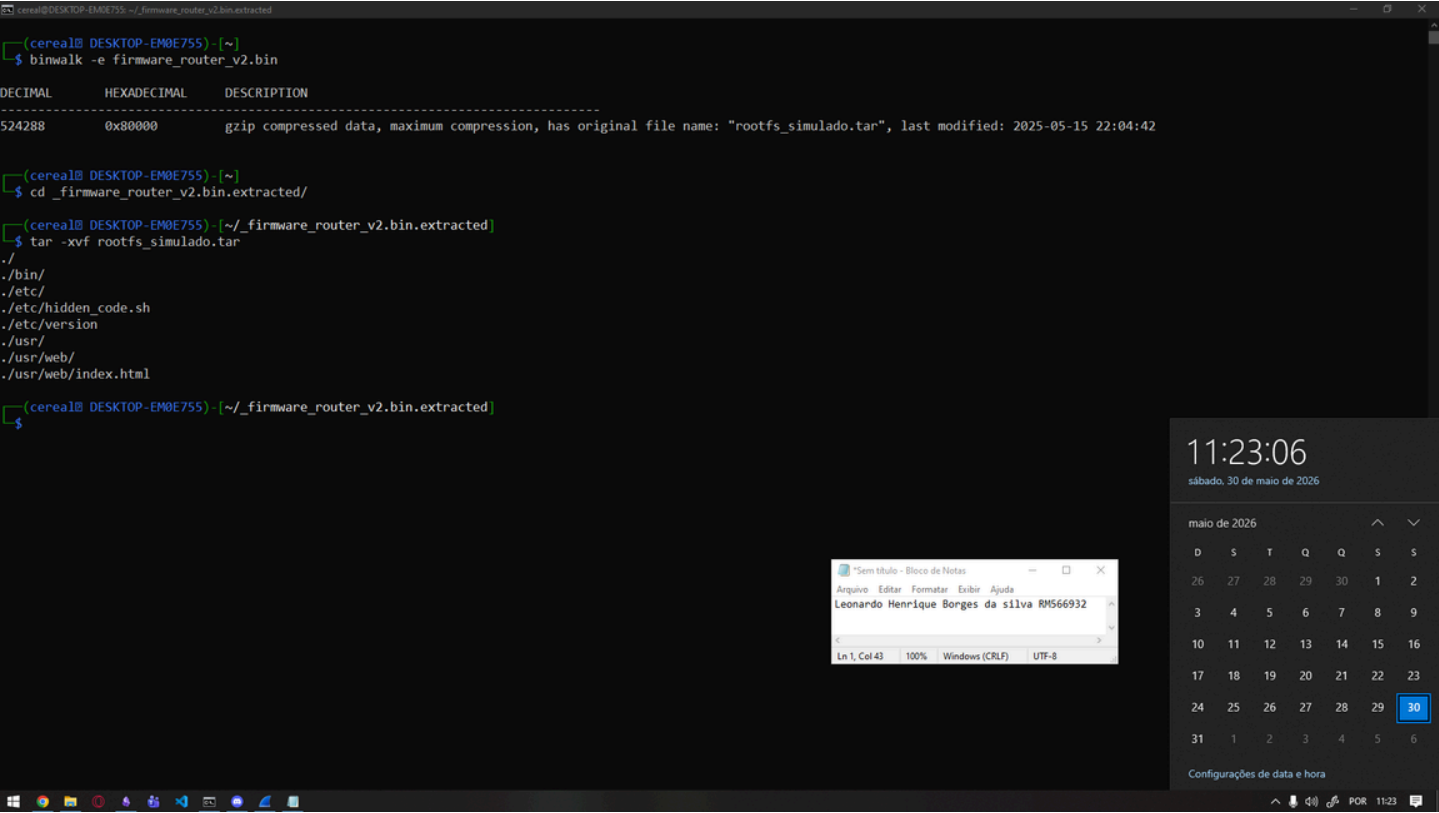
Em seguida, foi realizada a extração do conteúdo utilizando o Binwalk:

```
binwalk -e firmware_router_v2.bin
```

Durante o processo de extração foi necessário executar a ferramenta em um diretório com permissões adequadas devido a restrições de escrita encontradas no sistema de arquivos utilizado inicialmente.

Após a extração, foi possível acessar a estrutura interna do firmware e analisar os arquivos presentes na partição raiz.

Figura 17 – Estrutura de diretórios extraída do firmware



Durante a inspeção dos arquivos foi localizada a informação oculta utilizada para compor a flag do desafio.

Figura 18 – Localização da flag nos arquivos extraídos

```
cereal@DESKTOP-EM0E755: ~/_firmware_router_v2.bin.extracted/etc
(cereal@DESKTOP-EM0E755) ~$ binwalk -e firmware_router_v2.bin

DECIMAL      HEXADECIMAL    DESCRIPTION
-----
524288      0x80000      gzip compressed data, maximum compression, has original file name: "rootfs_simulado.tar", last modified: 2025-05-15 22:04:42

(cereal@DESKTOP-EM0E755) ~$ cd _firmware_router_v2.bin.extracted/

(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted$ tar -xvf rootfs_simulado.tar
./
./bin/
./etc/
./etc/hidden_code.sh
./etc/version
./usr/
./usr/web/
./usr/web/index.html

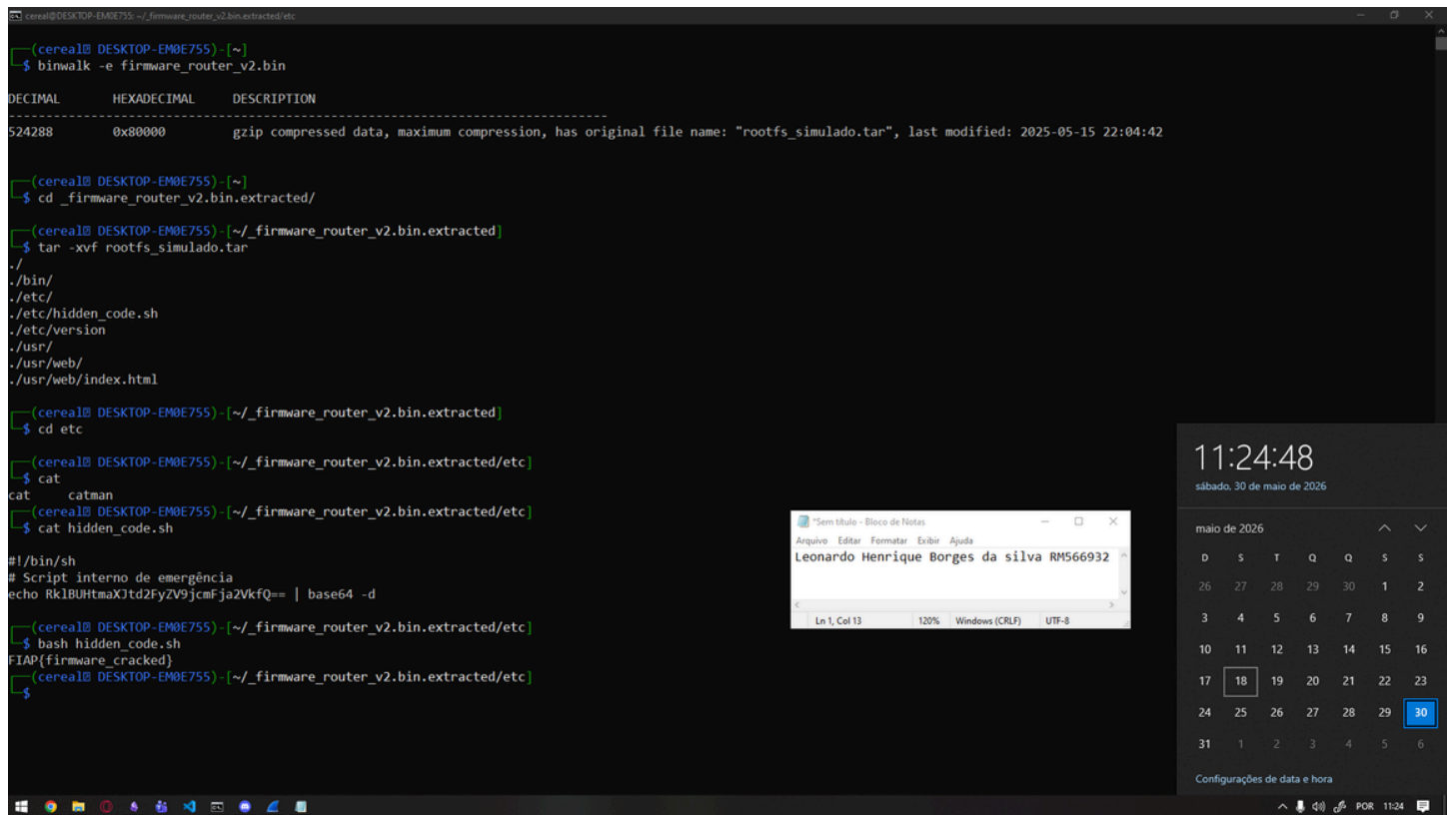
(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted$ cd etc

(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted/etc$ cat
cat      catman
(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted/etc$ cat hidden_code.sh

#!/bin/sh
# Script interno de emergência
echo Rk1BUHtmaXJtd2FyZV9jcmFja2VkfQ== | base64 -d

(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted/etc$ bash hidden_code.sh
FIAP{firmware_cracked}

(cereal@DESKTOP-EM0E755) ~/_firmware_router_v2.bin.extracted/etc$
```



Resultado

Através da extração e análise do sistema de arquivos presente no firmware foi possível localizar o código de acesso escondido pelos engenheiros.

A informação encontrada permitiu recuperar a flag solicitada pelo desafio e concluir a atividade com sucesso.

Flag obtida:

FIAP{firmware_cracked}

O desafio demonstrou técnicas utilizadas em análise de firmware e sistemas embarcados, evidenciando a importância da inspeção detalhada dos arquivos presentes em partições internas durante atividades de engenharia reversa e análise forense.

3.9 Firmware Embarcado V5: Fragmentos Invisíveis

Enunciado

Durante uma operação de engenharia reversa, um firmware (versão 5) foi extraído de um equipamento industrial de rede utilizado em ambientes isolados. A investigação aponta que engenheiros esconderam uma flag secreta dentro da interface web embarcada, de forma a evitar que ela fosse descoberta facilmente por análises simples.

O firmware já foi parcialmente extraído, e sua estrutura interna revela múltiplas páginas HTML. O conteúdo, aparentemente inofensivo, esconde algo mais sofisticado: a flag foi fragmentada.

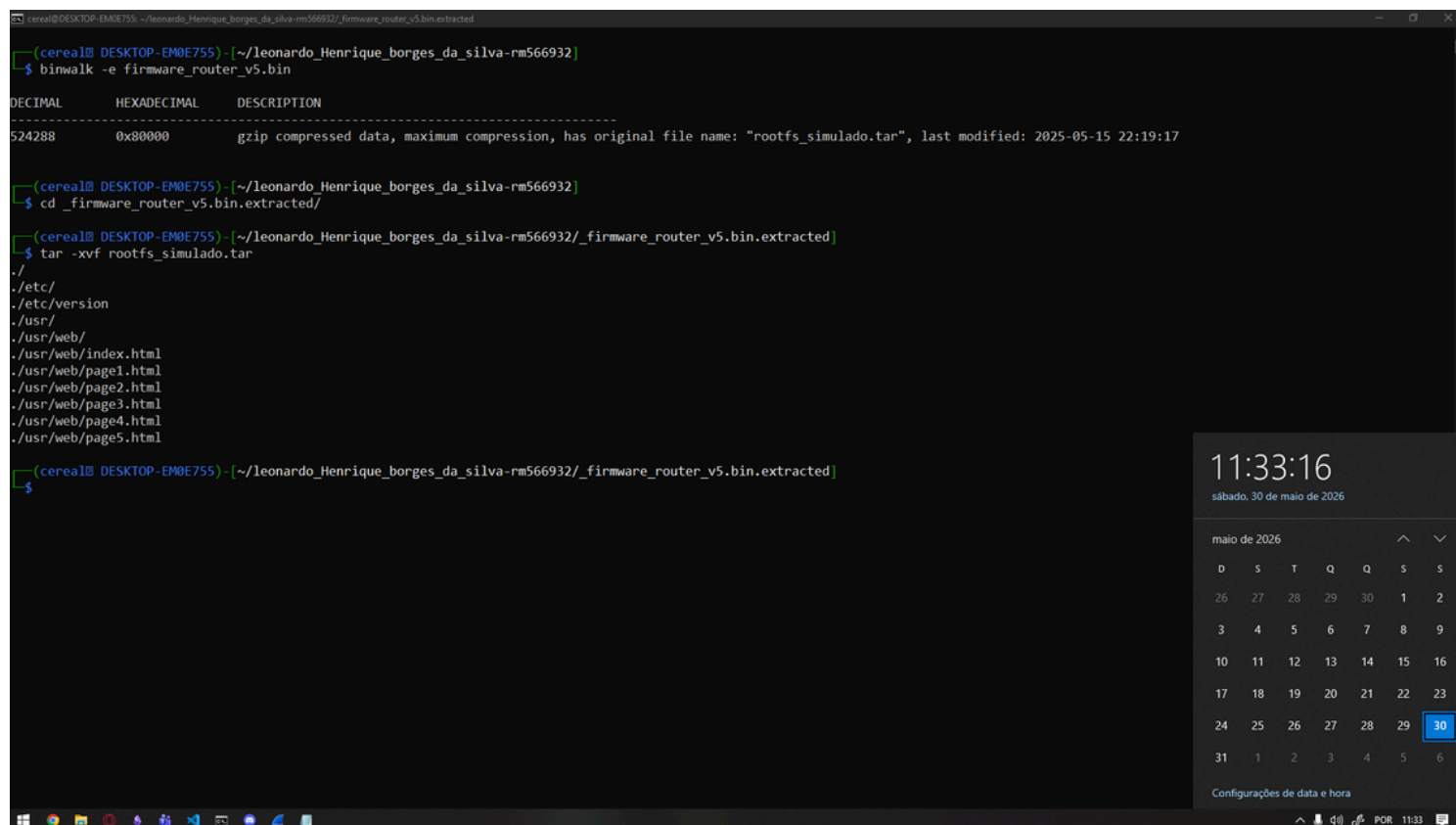
Seu objetivo: reconstruir a flag completa.

Arquivo: firmware_router_v5.bin

Metodologia

Inicialmente foi realizada a extração do firmware disponibilizado pelo desafio. Após a análise da estrutura de diretórios, foram identificados diversos arquivos HTML armazenados no diretório da interface web embarcada.

Figura 19 – Estrutura de arquivos da interface web embarcada



```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932/_firmware_router_v5.bin.extracted
(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ binwalk -e firmware_router_v5.bin

DECIMAL      HEXADECIMAL  DESCRIPTION
-----
524288      0x80000      gzip compressed data, maximum compression, has original file name: "rootfs_simulado.tar", last modified: 2025-05-15 22:19:17

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ cd _firmware_router_v5.bin.extracted/

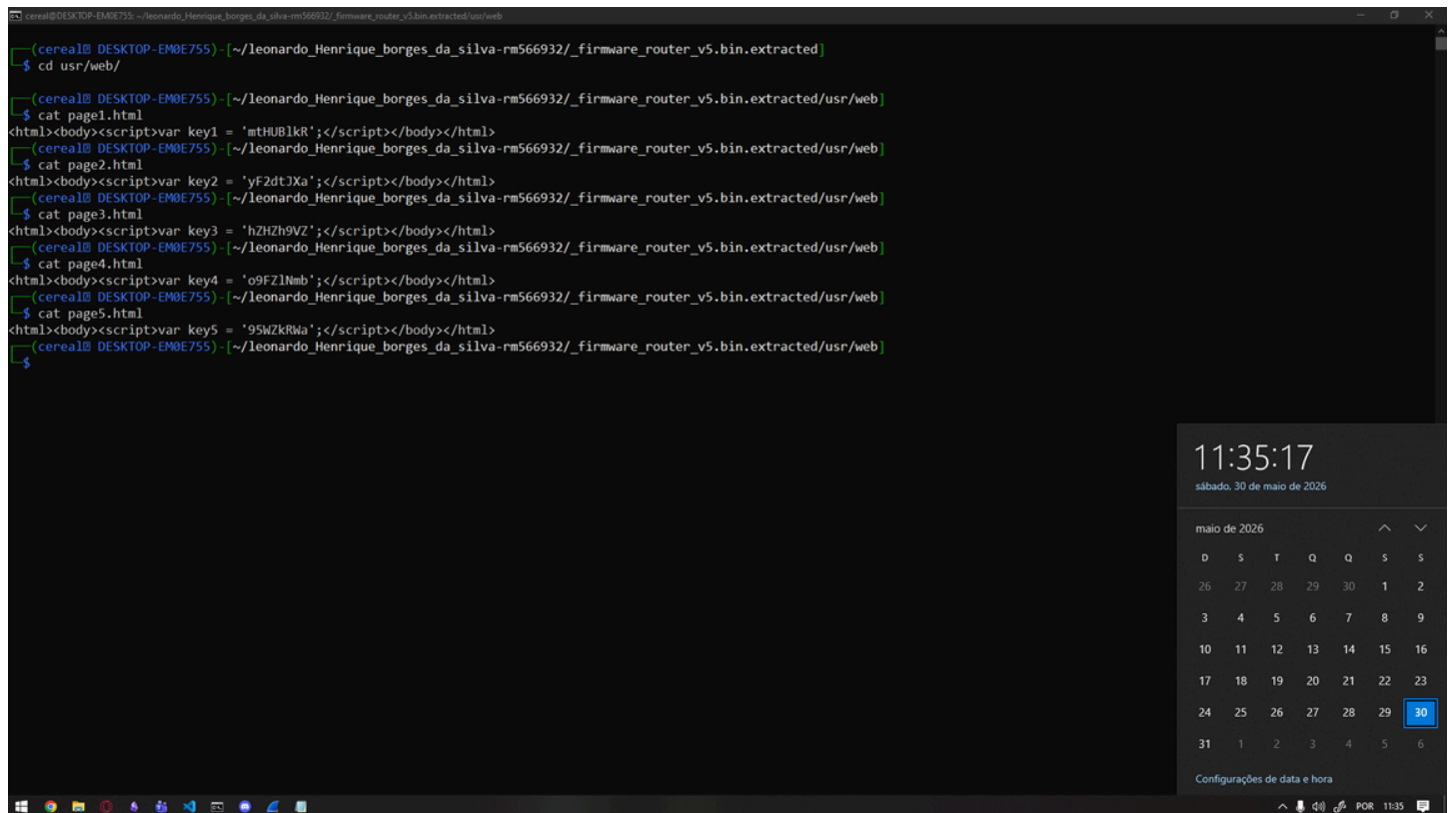
(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932/_firmware_router_v5.bin.extracted
$ tar -xvf rootfs_simulado.tar
./
./etc/
./etc/version
./usr/
./usr/web/
./usr/web/index.html
./usr/web/page1.html
./usr/web/page2.html
./usr/web/page3.html
./usr/web/page4.html
./usr/web/page5.html

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932/_firmware_router_v5.bin.extracted
$
```

The screenshot shows a terminal window with a Windows taskbar at the bottom. The terminal displays the execution of binwalk to extract a firmware file, followed by a directory listing of the extracted files. The files include a directory structure with /etc/ and /usr/web/ containing several HTML files. A system clock and calendar are visible in the bottom right corner of the terminal window.

Durante a inspeção do conteúdo das páginas foi identificado que cada arquivo armazenava um fragmento de uma sequência codificada.

Figura 20 – Fragmentos encontrados nos arquivos HTML

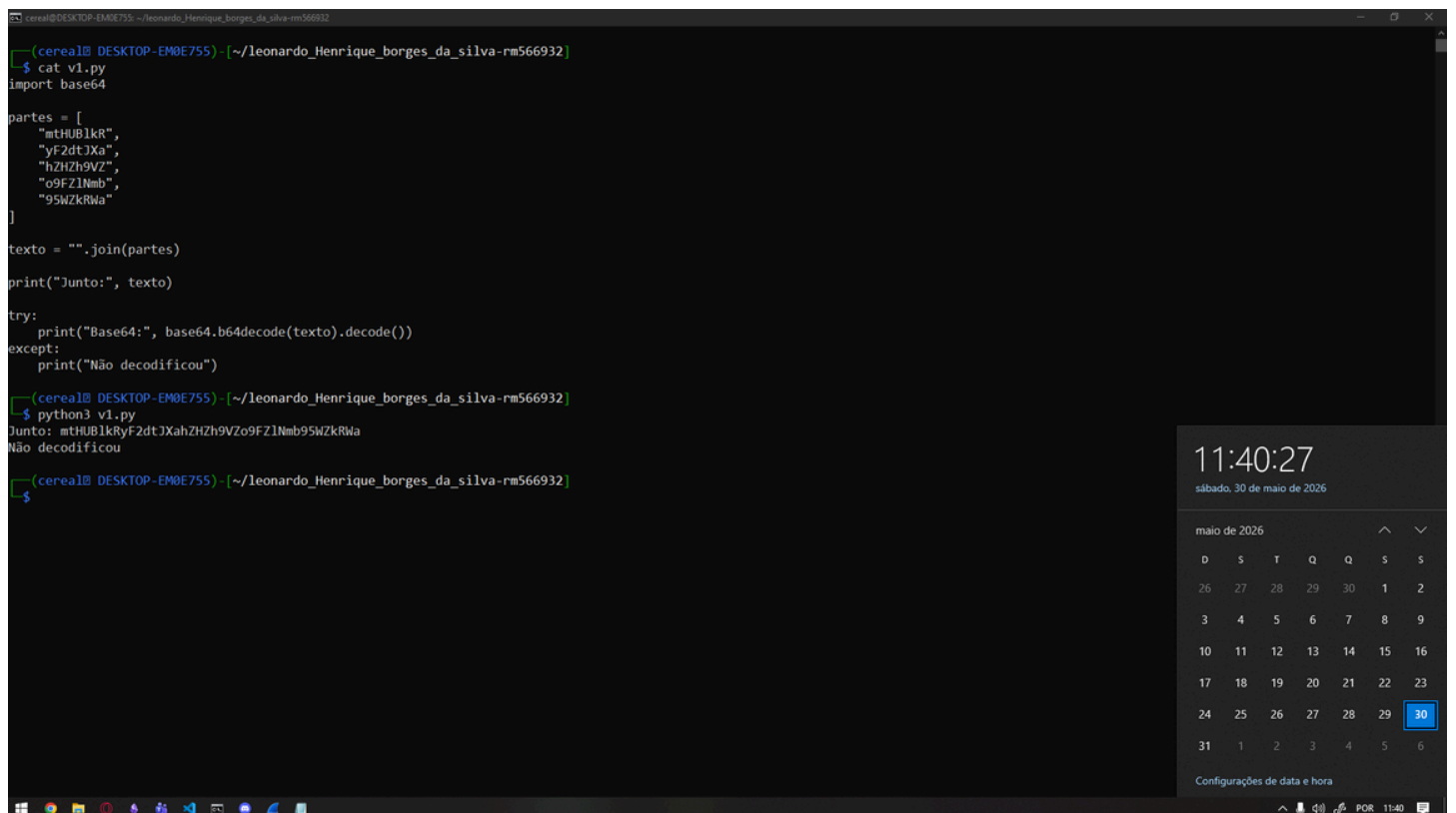


```
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted]
$ cd usr/web/

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$ cat page1.html
<html><body><script>var key1 = 'mtHUBlkR';</script></body></html>
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$ cat page2.html
<html><body><script>var key2 = 'yF2dtjXa';</script></body></html>
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$ cat page3.html
<html><body><script>var key3 = 'hZHZh9VZ';</script></body></html>
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$ cat page4.html
<html><body><script>var key4 = 'o9FZlNmb';</script></body></html>
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$ cat page5.html
<html><body><script>var key5 = '95WZkRWa';</script></body></html>
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932_firmware_router_v5.bin.extracted/usr/web]
$
```

Após coletar todos os fragmentos, foi desenvolvido um script em Python para automatizar o processo de reconstrução da sequência.

Figura 21 – Processo de reconstrução da sequência codificada



```
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat v1.py
import base64

partes = [
    "mtHUBlkR",
    "yF2dtjXa",
    "hZHZh9VZ",
    "o9FZlNmb",
    "95WZkRWa"
]

texto = "".join(partes)

print("Junto:", texto)

try:
    print("Base64:", base64.b64decode(texto).decode())
except:
    print("Não decodificou")

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ python3 v1.py
Junto: mtHUBlkRyF2dtjXahZHZh9VZo9FZlNmb95WZkRWa
Não decodificou

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```


Inversão de strings:

```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
$ nano v2.py

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ cat v2.py
import base64

s = "mtHUBlkRyF2dtJXahZHh9VZo9FZlNmb95WZkRWa"

inv = s[::-1]

print(inv)

try:
    print(base64.b64decode(inv + "=="))
except Exception as e:
    print(e)

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ python3 v2.py
aMRkZw59bmNlZF9oZV9hZHh9VZo9FZlNmb95WZkRWa

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$
```

11:41:09

sábado, 30 de maio de 2026

maio de 2026

D	S	T	Q	Q	S	S
26	27	28	29	30	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Configurações de data e hora

Rotacionado:

```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
$ cat v3.py
import base64

# Fragmentos encontrados
partes = [
    "mtHUBlkR",
    "yF2dtJXa",
    "hZHh9VZ",
    "o9FZlNmb",
    "95WZkRWa"
]

# 1. Junta tudo
texto = ''.join(partes)

print("[+] Junto:")
print(texto)

# 2. Inverte
invertido = texto[::-1]

print("\n[+] Invertido:")
print(invertido)

# 3. Base64 decode
decodificado = base64.b64decode(invertido + "==").decode()

print("\n[+] Decodificado:")
print(decodificado)

# 4. Reorganiza para começar com FIAP
idx = decodificado.index("FIAP")

flag_parcial = decodificado[idx:] + decodificado[:idx]

print("\n[+] Rotacionado:")
print(flag_parcial)

(cereal@DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ python3 v3.py
[+] Junto:
mtHUBlkRyF2dtJXahZHh9VZo9FZlNmb95WZkRWa

[+] Invertido:
aMRkZw59bmNlZF9oZV9hZHh9VZo9FZlNmb95WZkRWa

[+] Decodificado:
idden)nced_he_advairmwarFIAP{f

[+] Rotacionado:
FIAP{fidden)nced_he_advairmwar
```

11:42:40

sábado, 30 de maio de 2026

maio de 2026

D	S	T	Q	Q	S	S
26	27	28	29	30	1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31	1	2	3	4	5	6

Configurações de data e hora

Possíveis padrões:

```
cereal@DESKTOP-EMBE755: ~/leonardo_henrique_borges_da_silva-rm566932
--(cereal@DESKTOP-EMBE755) ~/leonardo_henrique_borges_da_silva-rm566932
--$ cat v5.py
import base64
import itertools

partes = [
    "mthUB1kR",
    "yf2dtJXa",
    "hZH2n9VZ",
    "c9fZlWmb",
    "9SNZkRMa"
]

# Junta fragmentos
texto = ''.join(partes)

# Inverte
invertido = texto[::-1]

# Base64 decode
decodificado = base64.b64decode(invertido + "==").decode()

# Rotaciona para começar em FIAP
idx = decodificado.index("FIAP")
rotacionado = decodificado[idx:] + decodificado[:idx]

print("[+] Resultado bruto:")
print(rotacionado)

# Palavras suspeitas do contexto
palavras = ["firmware", "advanced", "hidden"]

print("\n[+] Tentando combinações...\n")

# Testa combinações
for p in itertools.permutations(palavras):
    tentativa = f"FIAP{{{p[0]}.{p[1]}.{p[2]}}}"

    # heurística simples:
    # verifica se partes das palavras aparecem embaralhadas
    score = 0

    for palavra in p:
        for letra in palavra:
            if letra in rotacionado:
                score += 1

    # Se score alto, considera válida
    if score > 20:
        print("[+] Possível flag:")
        print(tentativa)

--(cereal@DESKTOP-EMBE755) ~/leonardo_henrique_borges_da_silva-rm566932
--$
```

11:44:32

sábado, 30 de maio de 2026

maio de 2026

D	S	T	Q	S	S
26	27	28	29	30	1
3	4	5	6	7	8
10	11	12	13	14	15
17	18	19	20	21	22
24	25	26	27	28	29
31	1	2	3	4	5

Configurações de data e hora

```
cereal@DESKTOP-EMBE755: ~/leonardo_henrique_borges_da_silva-rm566932
--(cereal@DESKTOP-EMBE755) ~/leonardo_henrique_borges_da_silva-rm566932
--$ python3 v5.py
[+] Resultado bruto:
FIAP{f1dden}nced_he_adva1rmuar

[+] Tentando combinações...

[+] Possível flag:
FIAP{firmware_advanced_hidden}
[+] Possível flag:
FIAP{firmware_hidden_advanced}
[+] Possível flag:
FIAP{advanced_firmware_hidden}
[+] Possível flag:
FIAP{advanced_hidden_firmware}
[+] Possível flag:
FIAP{hidden_firmware_advanced}
[+] Possível flag:
FIAP{hidden_advanced_firmware}

--(cereal@DESKTOP-EMBE755) ~/leonardo_henrique_borges_da_silva-rm566932
--$
```

A análise do resultado demonstrou que o conteúdo havia sido embaralhado propositalmente. Após reorganização lógica dos termos encontrados, foi possível reconstruir a frase original utilizada na flag.

Resultado

A reconstrução dos fragmentos distribuídos entre as páginas HTML permitiu recuperar a flag escondida na interface web embarcada.

Flag obtida:

FIAP{firmware_advanced_hidden}

O desafio demonstrou técnicas de análise de firmware, inspeção de aplicações web embarcadas e reconstrução de informações fragmentadas distribuídas em múltiplos arquivos.

3.10 Comunicação Ofuscada em Rede

Enunciado

Durante a investigação de um tráfego HTTP suspeito, foi capturado um pacote contendo uma mensagem, camuflada como um comentário técnico em um fluxo aleatório.

Objetivo: Recuperar a flag embutida no tráfego.

Arquivo: pcap_flag.pcap

Metodologia

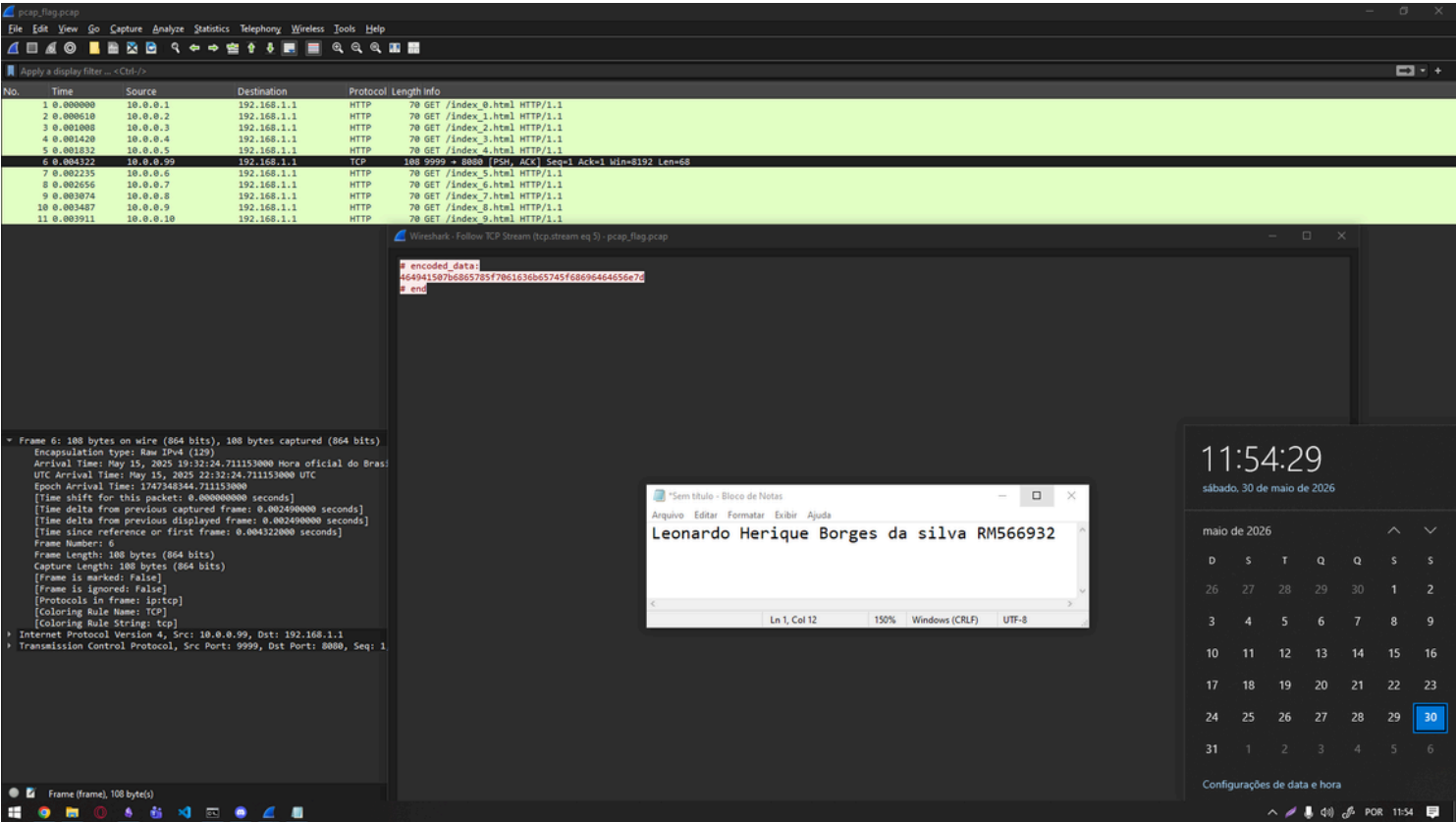
Inicialmente foi aberto o arquivo de captura utilizando o Wireshark para inspeção dos pacotes HTTP presentes na comunicação.

Durante a análise dos pacotes foi identificado um campo denominado:

encoded_data

contendo uma sequência hexadecimal suspeita.

Figura 22 – Identificação do campo encoded_data na captura de rede



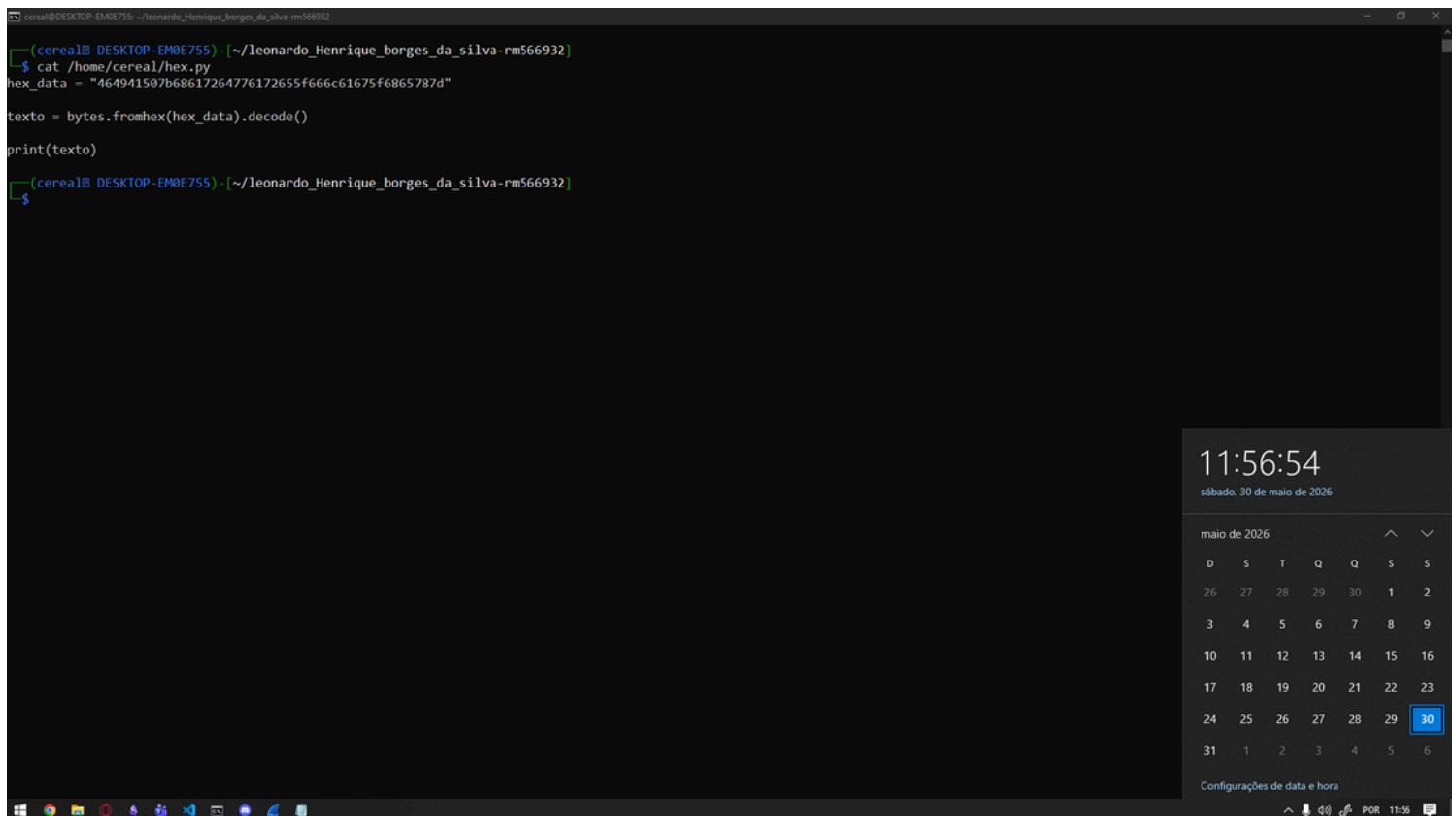
O conteúdo encontrado foi:

464941507b6865785f7061636b65745f68696464656e7d

Considerando que a sequência era composta exclusivamente por caracteres hexadecimais, levantou-se a hipótese de que a mensagem havia sido codificada em hexadecimal.

Para validar a hipótese, foi utilizada a linguagem Python para converter os dados hexadecimais em texto ASCII.

Script Utilizado



```
(cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932)
$ cat /home/cereal/hex.py
hex_data = "464941507b68617264776172655f6666c61675f6865787d"

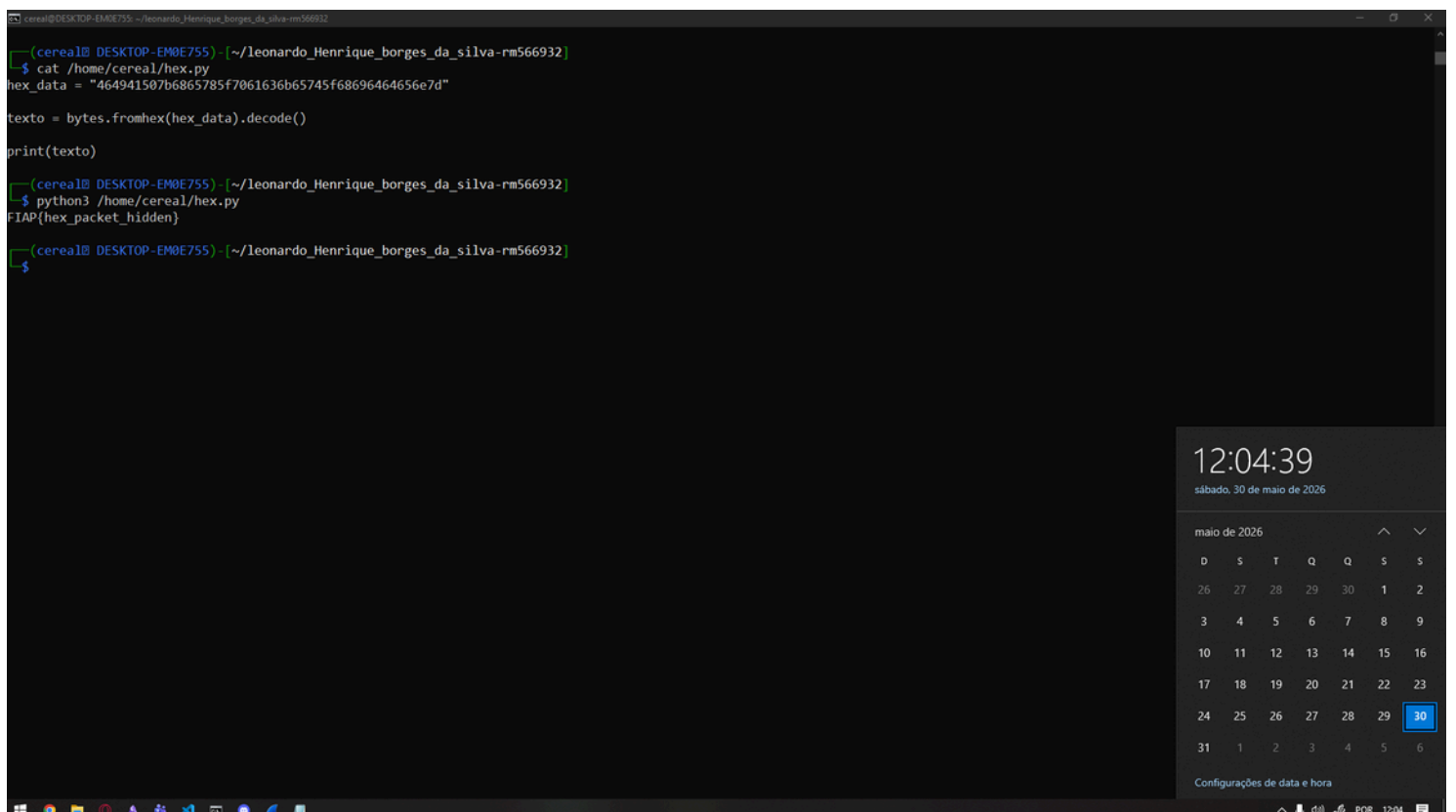
texto = bytes.fromhex(hex_data).decode()

print(texto)

(cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932)
$
```

The terminal window shows the execution of a Python script. The script defines a hexadecimal string, converts it to bytes, and then decodes it to ASCII text. The output is not visible in the screenshot. A system clock overlay on the right shows the time as 11:56:54 on Saturday, May 30, 2026.

Figura 23 – Conversão da sequência hexadecimal para texto ASCII



```
(cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932)
$ cat /home/cereal/hex.py
hex_data = "464941507b6865785f7061636b65745f68696464656e7d"

texto = bytes.fromhex(hex_data).decode()

print(texto)

(cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932)
$ python3 /home/cereal/hex.py
FIAP{hex_packet_hidden}

(cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932)
$
```

The terminal window shows the execution of a Python script. The script defines a hexadecimal string, converts it to bytes, and then decodes it to ASCII text. The output is visible as 'FIAP{hex_packet_hidden}'. A system clock overlay on the right shows the time as 12:04:39 on Saturday, May 30, 2026.

Após a execução do script, foi possível visualizar a mensagem originalmente transmitida dentro do tráfego HTTP.

Resultado

A análise do campo `encoded_data` demonstrou que a informação havia sido armazenada utilizando codificação hexadecimal simples.

Após a conversão para ASCII, foi possível recuperar diretamente a flag do desafio:

FIAP{hex_packet_hidden}

O desafio demonstrou a importância da inspeção detalhada de campos aparentemente comuns em protocolos de rede, bem como a utilização de técnicas básicas de codificação para ocultação de informações durante transmissões de dados.

3.11 Hardware Hacking EEPROM v7: Hex Oculto

Enunciado

Durante uma análise forense em um dispositivo embarcado interceptado, um dump binário da memória EEPROM foi obtido. Embora a maior parte do conteúdo pareça aleatória, analistas experientes notaram um pequeno padrão textual suspeito escondido entre blocos de ruído binário.

Qual é a flag oculta contida no dump binário da EEPROM?

Arquivo: eeprom_final_dump.bin

Metodologia

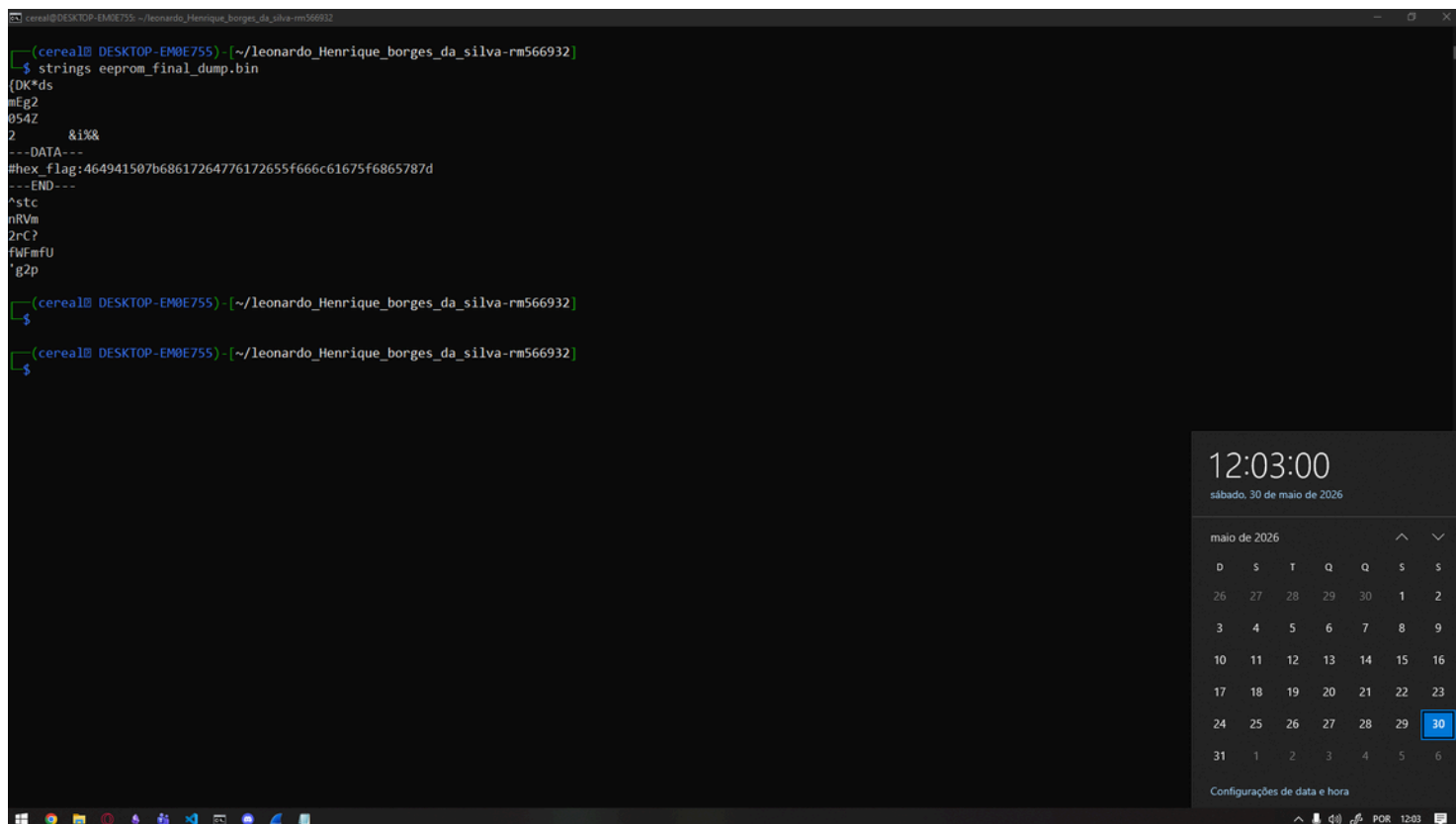
Inicialmente foi realizada uma análise do arquivo binário disponibilizado pelo desafio, buscando identificar padrões de texto legível em meio aos dados armazenados na EEPROM.

Considerando que a flag poderia estar armazenada em formato ASCII dentro do dump, foi utilizada a ferramenta strings, responsável por extrair sequências de caracteres legíveis presentes em arquivos binários.

O comando utilizado foi:

```
strings eeprom_v7.bin
```

Figura 24 – Extração de strings presentes no dump da EEPROM



```
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ strings eeprom_final_dump.bin
(DK*ds
mEg2
054Z
2
&1%&
---DATA---
#hex_flag:464941507b686172647761726555f666c61675f6865787d
---END---
^stc
nRVm
2rC?
fwFmFU
g2p

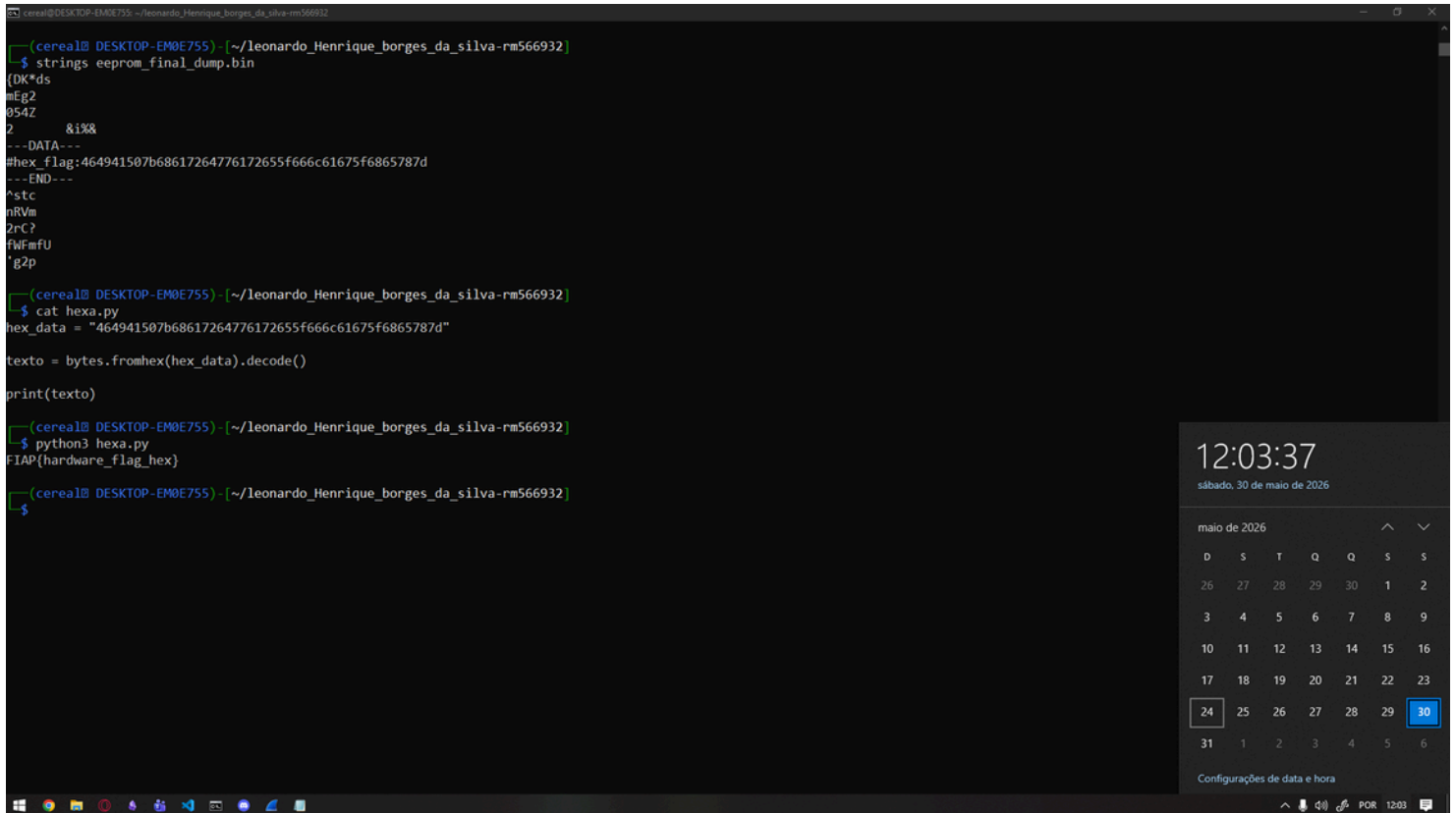
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

A saída retornada pela ferramenta foi analisada manualmente em busca de padrões compatíveis com o formato de flag adotado pelo desafio.

Durante a inspeção dos resultados foi identificada uma sequência textual suspeita escondida entre os dados extraídos.

Para facilitar a análise e validação da localização do conteúdo, também foi realizada uma inspeção hexadecimal do arquivo, permitindo visualizar a posição da informação dentro do dump.

Figura 25 – Visualização do conteúdo hexadecimal da EEPROM



```
(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ strings eeprom_final_dump.bin
{DK*ds
mEg2
054Z
2      8i%8
---DATA---
#hex_flag:464941507b686172647761726555f666c61675f6865787d
---END---
^stc
nRVm
2rC?
fWfMfU
'g2p

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat hexa.py
hex_data = "464941507b686172647761726555f666c61675f6865787d"

texto = bytes.fromhex(hex_data).decode()

print(texto)

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ python3 hexa.py
FIAP{hardware_flag_hex}

(cereal@DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Após a identificação da sequência textual, foi possível confirmar que ela correspondia à flag procurada.

Resultado

A análise do dump da EEPROM permitiu localizar uma sequência textual escondida em meio aos dados binários armazenados na memória do dispositivo.

A informação recuperada foi utilizada para obtenção da flag solicitada pelo desafio.

Flag obtida:

FIAP{hardware_flag_hex}

Este desafio demonstrou técnicas básicas de análise forense de memórias embarcadas, evidenciando como informações sensíveis podem permanecer armazenadas em dispositivos mesmo quando ocultas entre grandes volumes de dados binários.

3.12 Sinal Entre as Sombras

Descrição

Durante uma análise de tráfego de redes sem fio em um aeroporto internacional, um analista de segurança detectou uma atividade incomum: centenas de redes Wi-Fi foram identificadas em uma única varredura. Isso levanta a suspeita de um ataque de engenharia social ou de distração, como Evil Twin.

Sua missão é encontrar a rede suspeita.

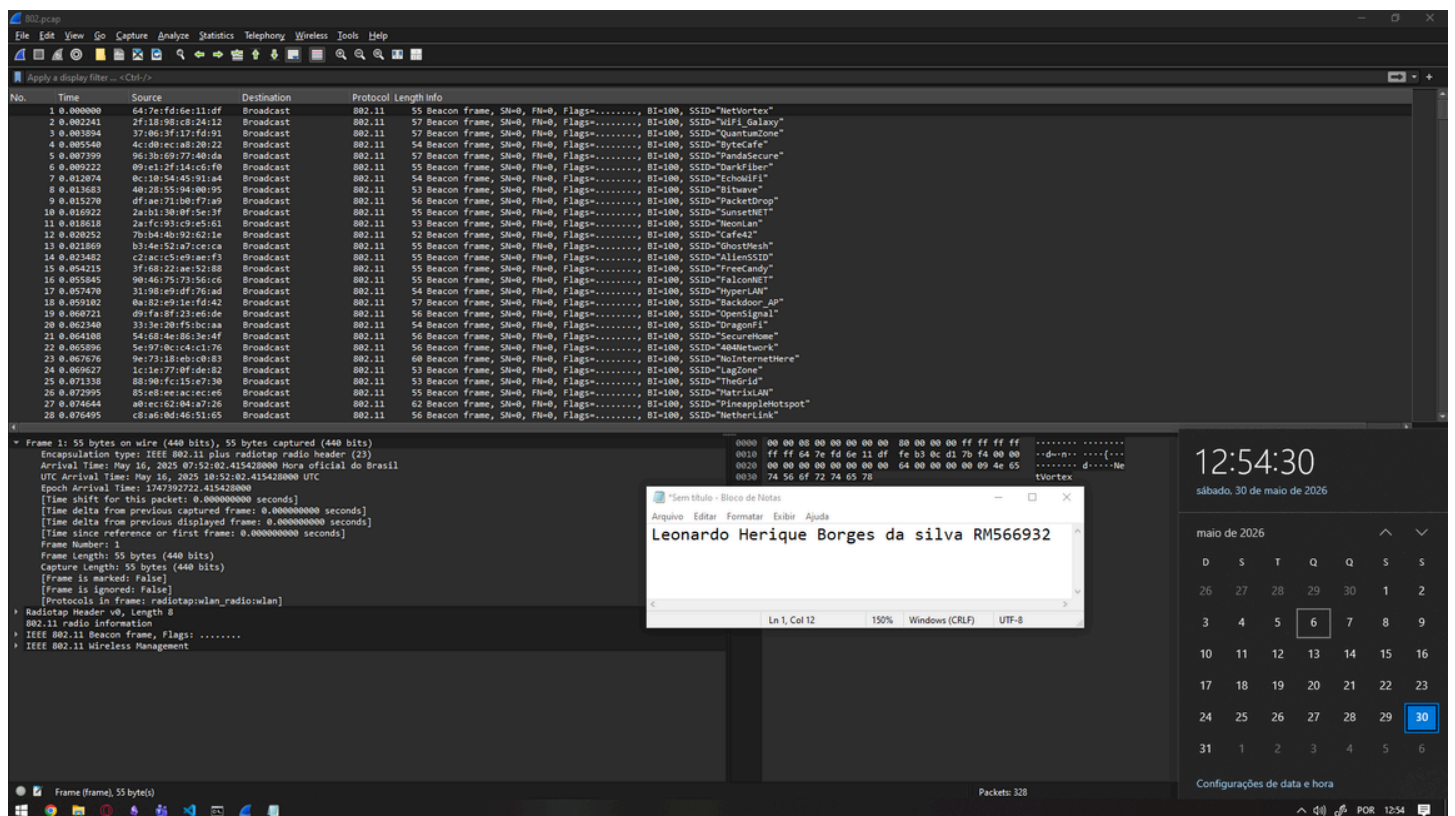
Arquivo: 802.pcap

Metodologia

Inicialmente foi realizada a análise do arquivo de captura utilizando a ferramenta Wireshark.

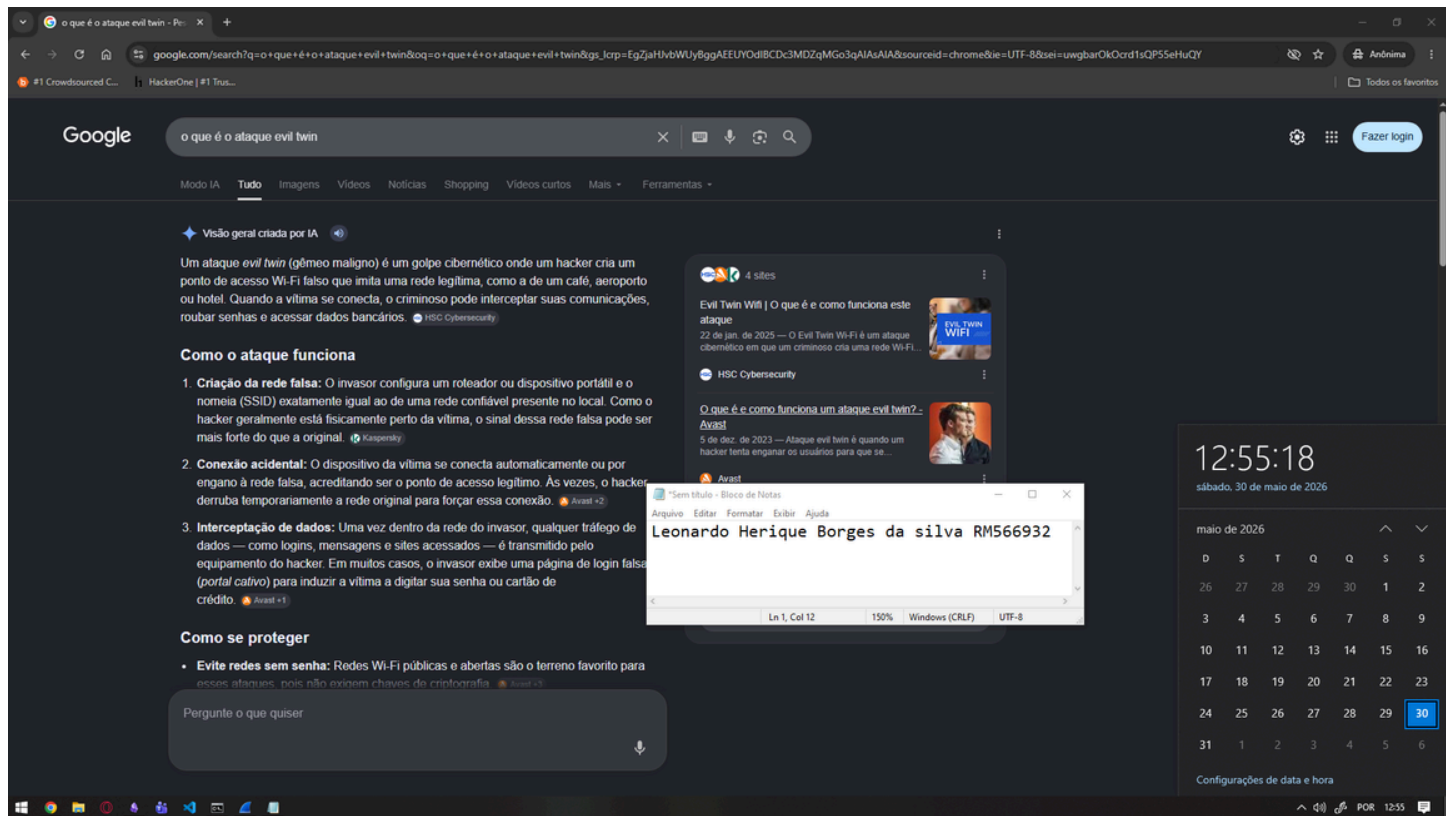
Durante a inspeção dos Beacon Frames foi possível observar uma grande quantidade de redes sem fio anunciadas.

Figura 25 – SSIDs encontrados na captura



Em seguida foi realizada uma pesquisa sobre ataques do tipo Evil Twin. Essa técnica consiste na criação de pontos de acesso falsos utilizando nomes semelhantes aos de redes legítimas com o objetivo de induzir usuários a se conectarem ao equipamento controlado pelo atacante.

Figura 26 – Pesquisa sobre a técnica Evil Twin



Considerando essa hipótese, foi desenvolvido um script em Bash para automatizar a análise dos SSIDs encontrados. O objetivo do script era:

- Extrair todos os SSIDs da captura;
- Converter os valores hexadecimais para texto legível;
- Identificar redes contendo referências à FIAP;
- Agrupar redes com nomes semelhantes que pudessem indicar tentativas de clonagem.

Script Utilizado

```
cereal@DESKTOP-EMBE755: ~/leonardo_henrique_borges_da_silva-rm566932
$ cat wlan2.sh
#!/bin/bash

ARQUIVO="/mnt/f/Downloads/B02.pcap"

echo "[*] Rodando o tshark... pera aí que demora um pouco"

AQUELA_LISTA_HEX=$(tshark -r "$ARQUIVO" -Y "wlan.fc.type_subtype == 0x08" -T fields -e wlan.ssid | grep -v "" | sort | uniq)

ma -f /tmp/ssids_puros.txt
touch /tmp/ssids_puros.txt

for x in $AQUELA_LISTA_HEX; do
    traduzido=$(echo "$x" | xxd -r -p)
    if [ ! -z "$traduzido" ]; then
        echo "$traduzido" >> /tmp/ssids_puros.txt
    fi
done

echo ""
echo "=====
echo "LISTA 1 - TODOS OS SSIDS DA REDE"
echo "=====
cat /tmp/ssids_puros.txt

echo ""
echo "=====
echo "LISTA 2 - SÓ O QUE TEM FIAP (ALVO)"
echo "=====
cat /tmp/ssids_puros.txt | grep -i "fiap"

echo ""
echo "=====
echo "LISTA 3 - SUSPEITOS DE EVIL TWIN (NOMES PERTO)"
echo "=====
echo "[*] Analizando possíveis clones..."
echo ""

cat /tmp/ssids_puros.txt | awk '{
    nome = $0
    # deixa minusculo pra nao dar ruim no grep
    limpo = tolower(nome)
    gsub(/["a-z]/, "", limpo)

    # pega as primeiras 3 ou 4 letras do SSID
    comeco = substr(limpo, 1, 4)

    if (comeco != "") {
        grupo[comeco] = grupo[comeco] " | " nome
        qtd[comeco]++
    }
}

END {
    for (g in grupo) {
        if (qtd[g] > 1) {
            print "Prováveis redes ->" grupo[g]
            print "=====
        }
    }
}'

ma -f /tmp/ssids_puros.txt

cereal@DESKTOP-EMBE755: ~/leonardo_henrique_borges_da_silva-rm566932
$
```

Figura 27 – Execução do script de análise dos SSIDs

```
Selecionar cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
(cereal@DESKTOP-EM0E755)~$ bash wlan2.sh
[*] Rodando o tshark... pera aí que demora um pouco

=====
LISTA 1 - TODOS OS SSIDS DA REDE
=====
404LAN
404Network
APocalypse
ARPPAttack
AccessDenied
AdminMode
AdminOnly
AirCrackTest
AirborneLAN
AlienSSID
AnonBox
AnonSignal
AntiSocialWifi
BSSID_007
Backdoor_AP
BeaconBurst
BeaconTrap
BeaconValley
Bitwave
BlackBoxNet
BlackMirror
BlackholeAP
BlackoutNet
BoomLAN
BotNet_Entry
BotWifi
Bot_999
BountyHunterAP
BroadcastStorm
BufferFlood
ByteCafe
C0d3Fi
Cafe42
CaffeinatedLAN
CamouflageFi
CaveLAN
Click2Connect
WatchTower
WelcomeAgent
WelcomeToNSA
WhosThere
WiCrypt
WifiPolice
WifiTrap
Wifi_666
Wifi_Galaxy
Wifi_Inside
WiFly
WiFree
WiNot
WiPi
Wifiboom
WizardLAN
WololoWifi
XtremeLAN
YggdrasilNet
Z3nSSID
Z3r0Net
ZeroAccess
ZeroPing
ZeroPointLAN
ZeroShell
ZetaNet
ZeusSSID
ZionLAN
ZombieNet
fiap(wifi)
FreeHackZone
PingStorm
AlienMesh
CursedLAN
LAN_of_the_Brave
BSkyMesh

=====
LISTA 2 - SÓ O QUE TEM FIAP (ALVO)
=====
FiapHack
Fiap_Testing
fiap(wifi)
=====
```

```
LISTA 3 - SUSPEITOS DE EVIL TWIN (NOMES PERTO)
=====
[!] Analisando possíveis clones...

Provaveis redes -> | PingBuster | PingEcho | PinglessZone | PingStorm
Provaveis redes -> | ZeroAccess | ZeroPing | ZeroPointLAN | ZeroShell
Provaveis redes -> | SSID_HOLE | SSiDisturbed
Provaveis redes -> | WelcomeAgent | WelcomeToNSA
Provaveis redes -> | MoonNet | MoonRouter
Provaveis redes -> | StarbaseLAN | Starfall
Provaveis redes -> | PentestOnly | PentestPoint
Provaveis redes -> | NodeJunk | NodePoint
Provaveis redes -> | FastHack | FastHackLAN | FastNFree
Provaveis redes -> | AdminMode | AdminOnly
Provaveis redes -> | OverclockedNet | OverkillSSID
Provaveis redes -> | SnowCrashWiFi | SnowLAN
Provaveis redes -> | FireTrap | FirewallFi
Provaveis redes -> | DoomLAN | DoomWiFi
Provaveis redes -> | Hack3rSp4c3 | HackTheAir | HackerLounge | HackySSID
Provaveis redes -> | CryptNet | CryptoAir
Provaveis redes -> | RainLAN | RainbowNet | RainstormAP
Provaveis redes -> | BlackBoxNet | BlackMirror | BlackholeAP | BlackoutNet
Provaveis redes -> | PacketCrafter | PacketDrop | PacketStorm | PacketVoid
Provaveis redes -> | FiapHack | Fiap_Testing | fiap{wifi}
Provaveis redes -> | Silentian | SilentPacket | SilentStorm
```

Após a conversão dos SSIDs para texto legível e a filtragem das redes relacionadas à FIAP, foi identificada uma rede cujo nome correspondia diretamente ao formato utilizado pelas flags dos desafios.

Resultado

A análise dos SSIDs presentes na captura permitiu identificar a rede suspeita escondida entre centenas de anúncios Wi-Fi.

Portanto, a flag recuperada para o desafio foi:

FIAP{wifi}

Este desafio demonstrou técnicas de análise de tráfego Wi-Fi, identificação de SSIDs ocultos e investigação de cenários relacionados a ataques do tipo Evil Twin, utilizando automação para acelerar o processo de análise.

3.13 Pacotes Múltiplos

Enunciado

Você foi contratado para testar um sistema de detecção de redes falsas. Seu trabalho consiste em simular a presença de redes Wi-Fi usando beacon frames com SSIDs distintos, mas todas pertencendo ao mesmo ponto de acesso (BSSID):

Após gerar e salvar o arquivo .pcap, responda:

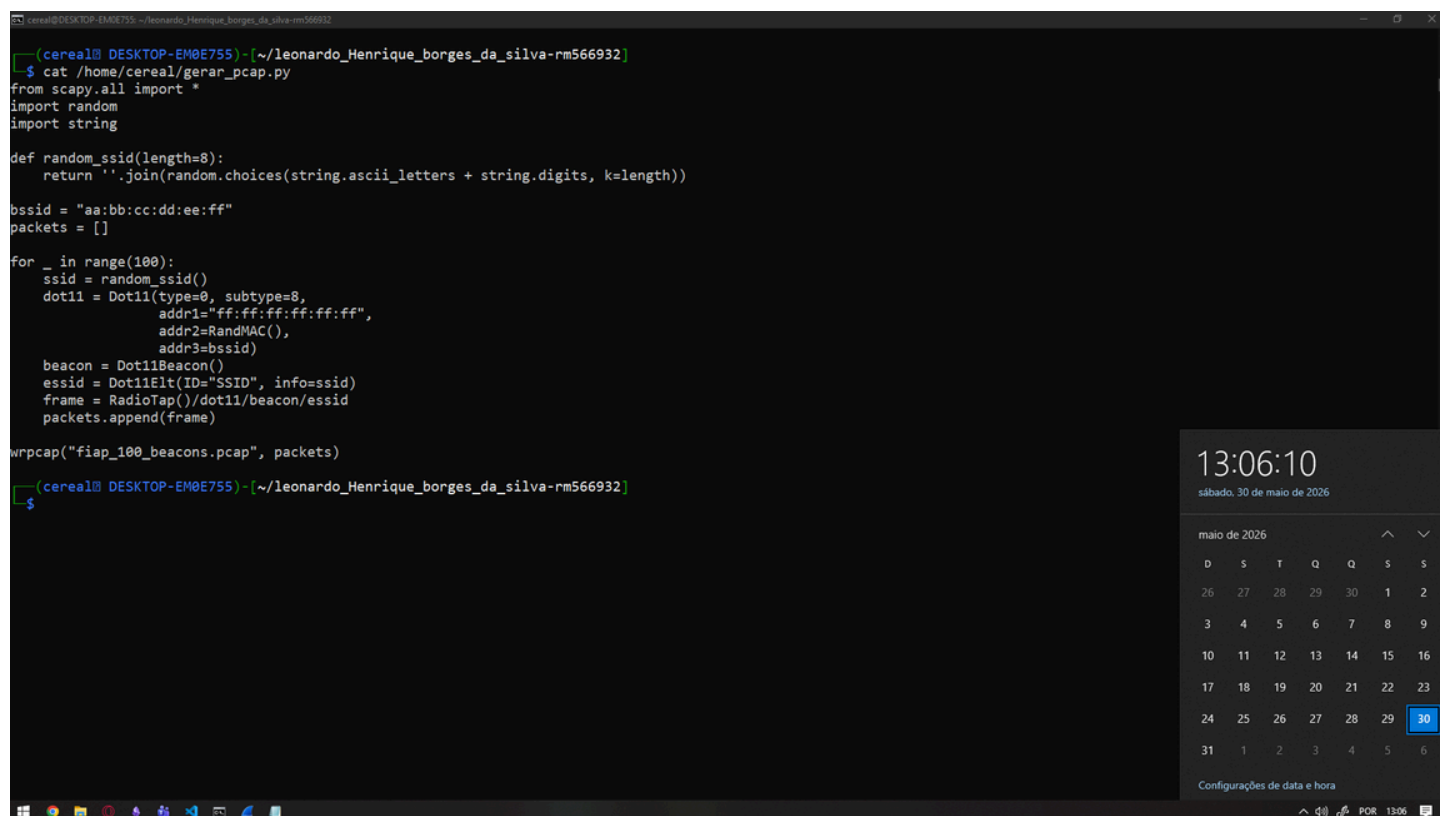
Quantos pacotes no arquivo possuem o BSSID aa:bb:cc:dd:ee:ff?

Arquivo: gerar_pcap.py

Metodologia

Inicialmente foi realizado o download do script Python disponibilizado pelo desafio. A execução desse script gerou um arquivo de captura de rede (.pcap) contendo os Beacon Frames utilizados na simulação.

Figura 28 – Script Python fornecido pelo desafio



```
(cereal@ DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ cat /home/cereal/gerar_pcap.py
from scapy.all import *
import random
import string

def random_ssid(length=8):
    return ''.join(random.choices(string.ascii_letters + string.digits, k=length))

bssid = "aa:bb:cc:dd:ee:ff"
packets = []

for _ in range(100):
    ssid = random_ssid()
    dot11 = Dot11(type=0, subtype=8,
                  addr1="ff:ff:ff:ff:ff:ff",
                  addr2=RandMAC(),
                  addr3=bssid)

    beacon = Dot11Beacon()
    essid = Dot11Elt(ID="SSID", info=ssid)
    frame = RadioTap()/dot11/beacon/essid
    packets.append(frame)

wrpcap("fiap_100_beacons.pcap", packets)

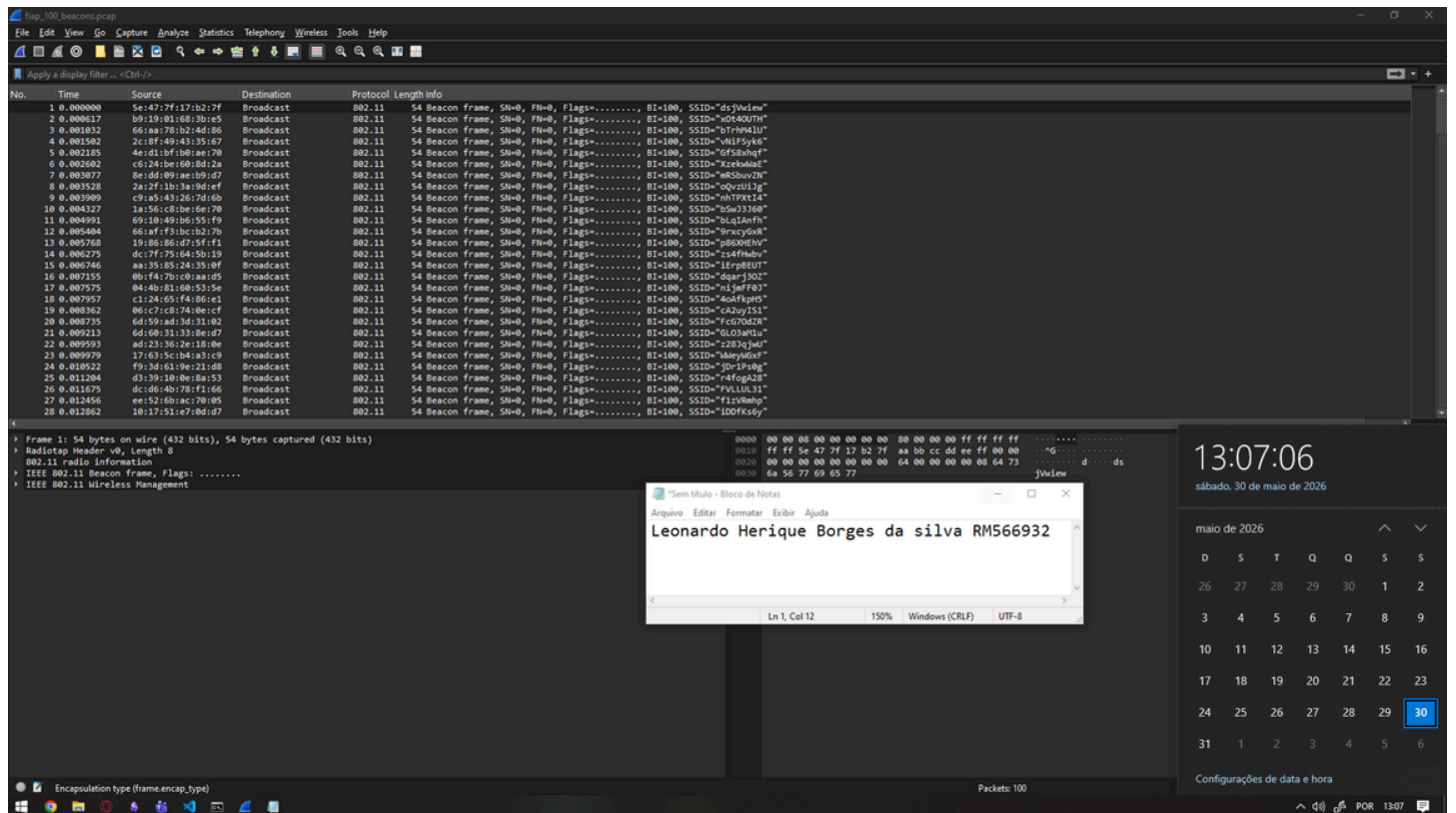
(cereal@ DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$
```

Após a geração do arquivo PCAP, a captura foi aberta utilizando o Wireshark para inspeção dos pacotes presentes.

Durante a análise dos Beacon Frames foi observado que todos os pacotes anunciavam SSIDs distintos, simulando a presença de diversas redes sem fio. Entretanto, ao inspecionar os detalhes dos quadros IEEE 802.11, verificou-se que todos utilizavam o mesmo BSSID:

aa:bb:cc:dd:ee:ff

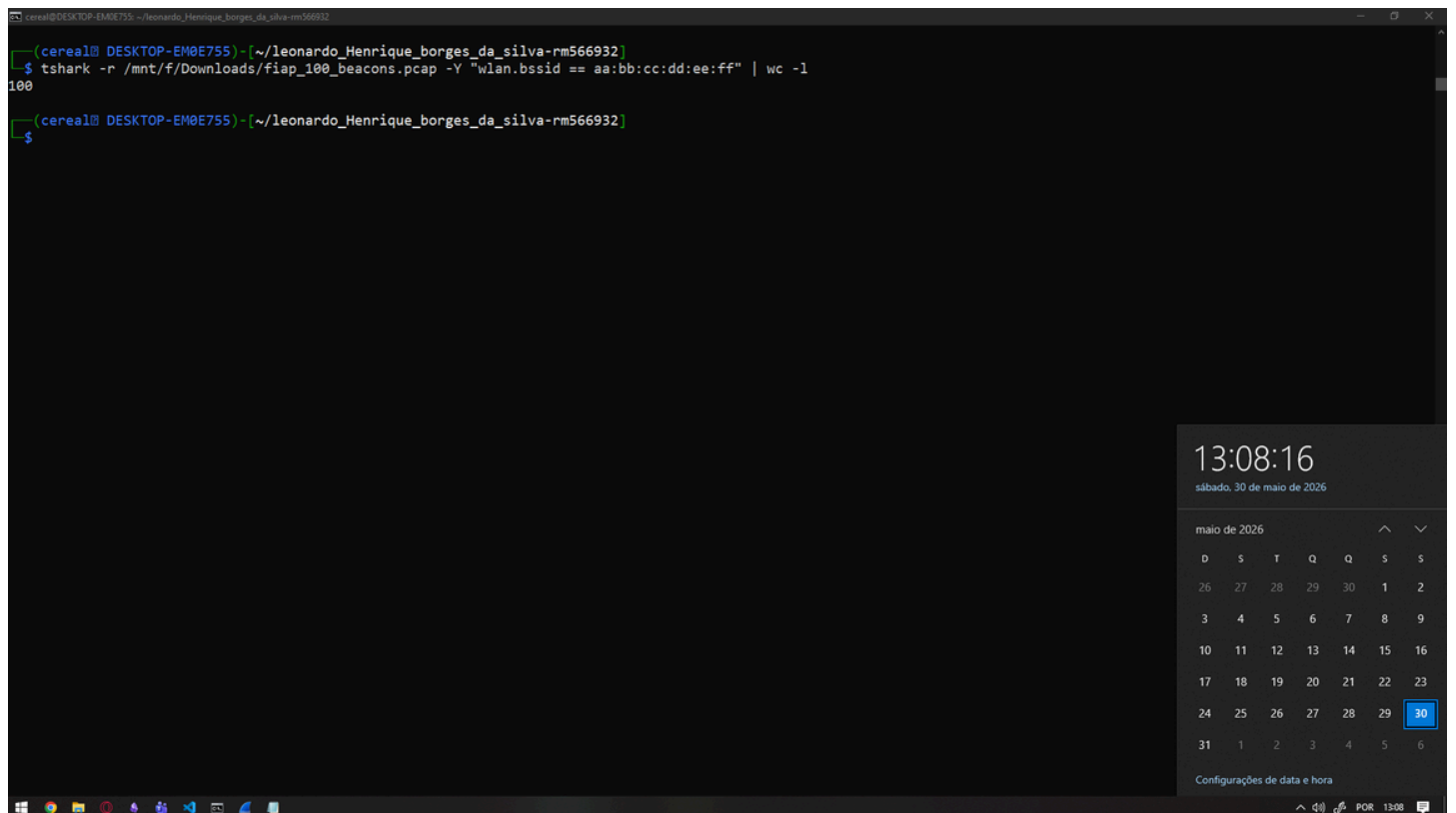
Figura 29 – Análise dos Beacon Frames no Wireshark



Para confirmar a quantidade de pacotes associados ao BSSID informado, foi aplicado um filtro no Wireshark exibindo apenas os quadros correspondentes ao endereço especificado.

A análise demonstrou que todos os Beacon Frames da captura pertenciam ao mesmo ponto de acesso, diferenciando-se apenas pelo SSID anunciado.

Figura 30 – Contagem dos pacotes associados ao BSSID analisado



Resultado

A inspeção da captura revelou que o arquivo continha exatamente 100 Beacon Frames associados ao BSSID informado no enunciado.

Portanto, a flag obtida foi:

FIAP{100}

Este desafio demonstrou conceitos relacionados ao funcionamento dos Beacon Frames em redes IEEE 802.11, evidenciando como um único ponto de acesso pode anunciar múltiplos SSIDs distintos utilizando o mesmo endereço físico (BSSID).

3.14 Shell Secreto

Enunciado

Um de nossos analistas encontrou um script em um servidor comprometido. O conteúdo do script parece inofensivo e não contém flag explícita, nem comentários com pistas.

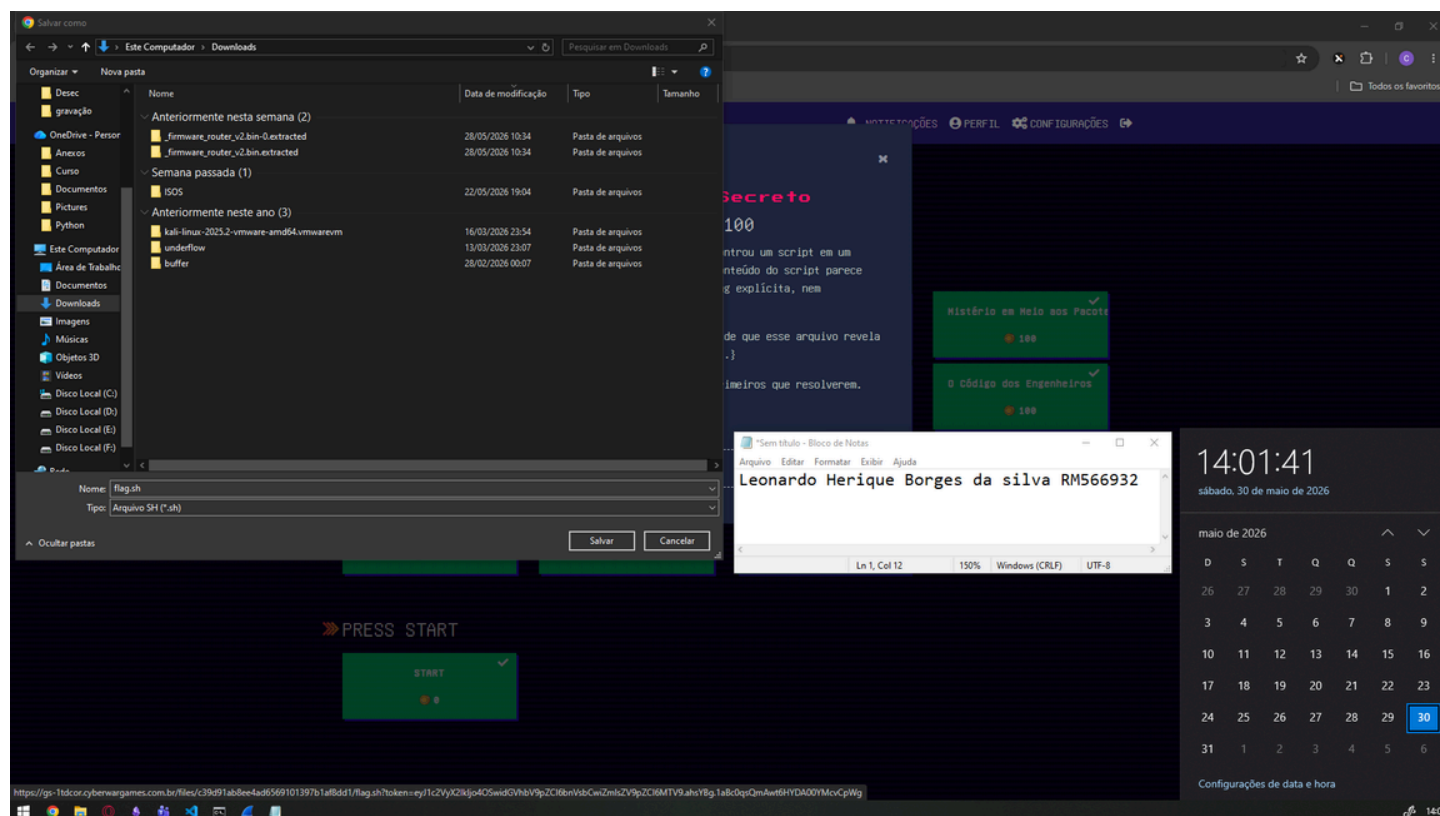
Entretanto, temos indícios de que esse arquivo revela uma flag no formato FIAP{...}

Arquivo: flag.sh

Metodologia

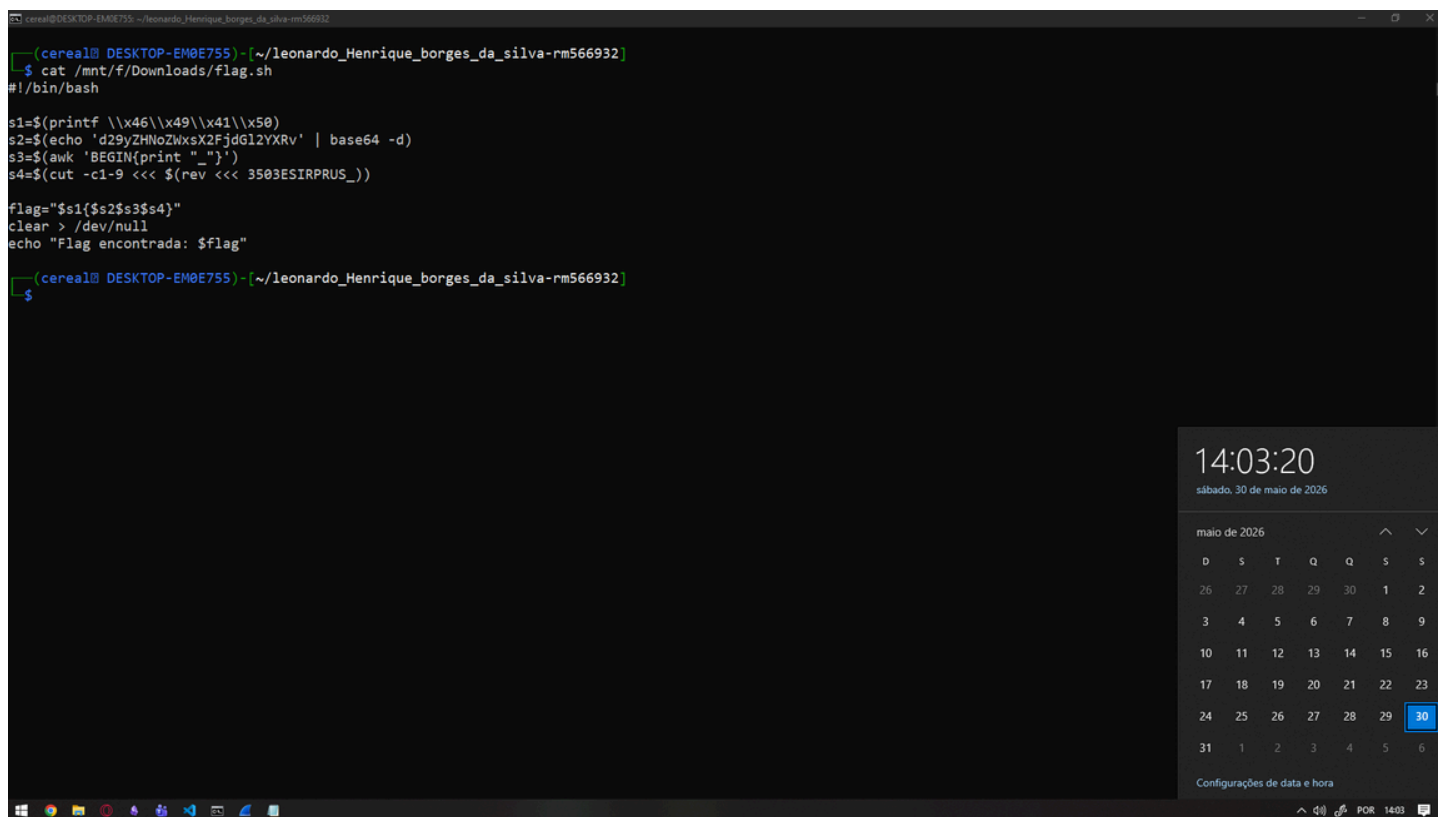
Inicialmente foi realizado o download do arquivo disponibilizado pelo desafio para análise local.

Figura 31 – Download do script disponibilizado pelo desafio



Após a obtenção do arquivo, foi realizada a inspeção de seu conteúdo e posterior execução em ambiente controlado.

Figura 32 – Conteúdo do script analisado



```
(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat /mnt/f/Downloads/flag.sh
#!/bin/bash

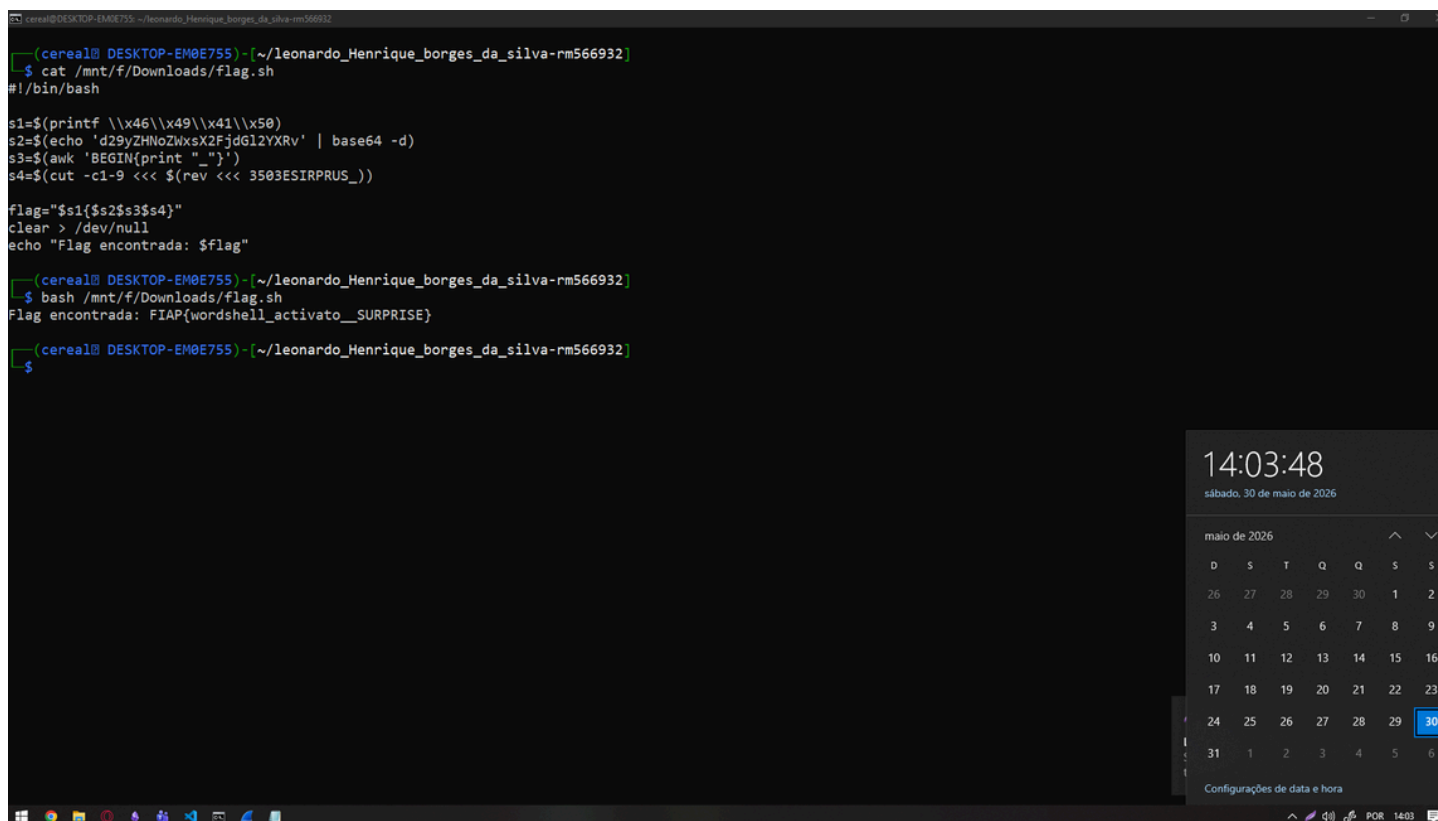
s1=$(printf '\x46\x49\x41\x50')
s2=$(echo 'd29yZHNoZWxsX2FjdG12YXRv' | base64 -d)
s3=$(awk 'BEGIN{print "_"}')
s4=$(cut -c1-9 <<< $(rev <<< 3503ESIRPRUS_))

flag="$s1${s2}${s3}${s4}"
clear > /dev/null
echo "Flag encontrada: $flag"

(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Ao executar o script, foi possível observar que uma string semelhante a uma flag era exibida. Entretanto, o resultado não seguia corretamente o formato esperado pelo desafio.

Figura 33 – Execução inicial do script



```
(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat /mnt/f/Downloads/flag.sh
#!/bin/bash

s1=$(printf '\x46\x49\x41\x50')
s2=$(echo 'd29yZHNoZWxsX2FjdG12YXRv' | base64 -d)
s3=$(awk 'BEGIN{print "_"}')
s4=$(cut -c1-9 <<< $(rev <<< 3503ESIRPRUS_))

flag="$s1${s2}${s3}${s4}"
clear > /dev/null
echo "Flag encontrada: $flag"

(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ bash /mnt/f/Downloads/flag.sh
Flag encontrada: FIAP{wordshell_activato__SURPRISE}

(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Durante a análise foi identificado que a flag era construída a partir da combinação de múltiplas técnicas de manipulação de texto:

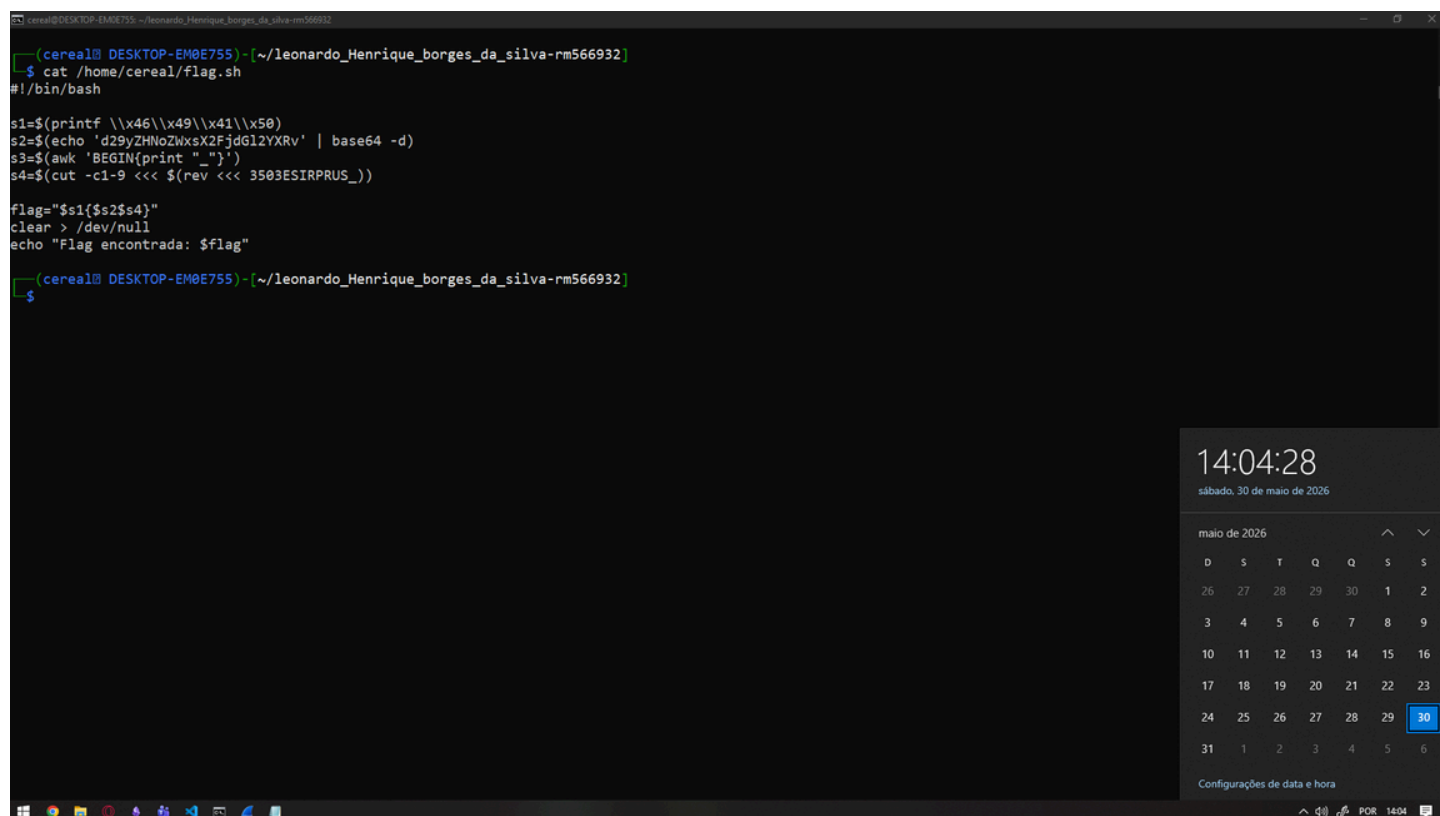
- Conversão de caracteres hexadecimal para ASCII;
- Decodificação Base64;

- Geração de caracteres utilizando AWK;
- Inversão de strings utilizando o comando rev.

Ao reconstruir manualmente as partes geradas pelo script, observou-se que havia um caractere _ excedente, impedindo a formação correta da flag.

Para validar a hipótese, foi criado um script simplificado reproduzindo a lógica original e corrigindo a concatenação dos elementos.

Figura 34 – Script utilizado para reconstrução da flag



```
(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat /home/cereal/flag.sh
#!/bin/bash

s1=$(printf '\x46\x49\x41\x50')
s2=$(echo 'd29yZHNoZWxsX2FjdG12YXRv' | base64 -d)
s3=$(awk 'BEGIN{print "_"}')
s4=$(cut -c1-9 <<< $(rev <<< 3503ESIRPRUS_))

flag="$s1$s2$s4"
clear > /dev/null
echo "Flag encontrada: $flag"

(cereal@ DESKTOP-EM0E755) [~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Após a correção da montagem da string, foi possível visualizar a flag completa no formato exigido pelo desafio.

Figura 35 – Resultado após correção da lógica do script

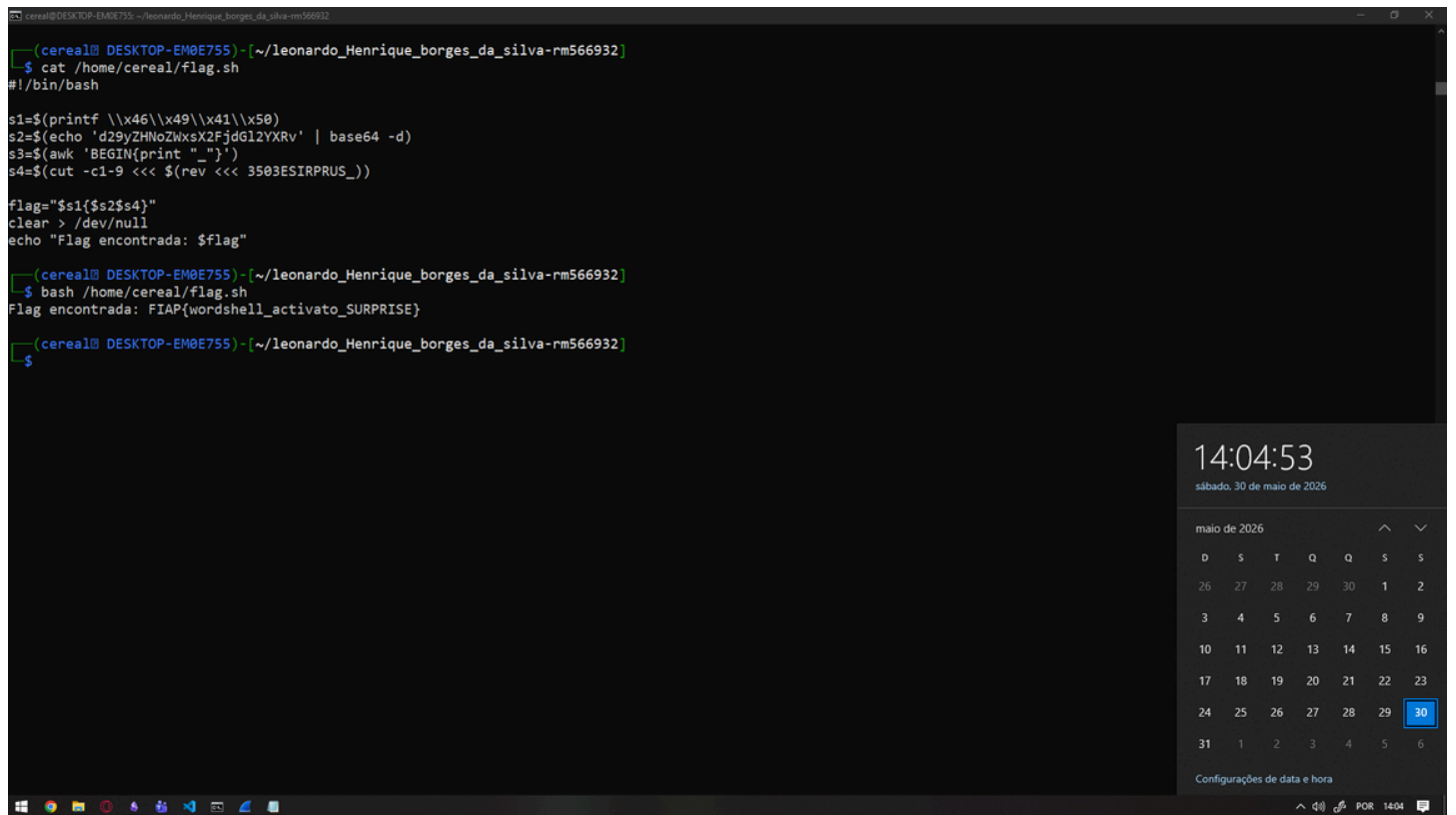
```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
(cereal@DESKTOP-EM0E755)~/leonardo_Henrique_borges_da_silva-rm566932
$ cat /home/cereal/flag.sh
#!/bin/bash

s1=$(printf '\x46\x49\x41\x50')
s2=$(echo 'd29yZHNoZWxsX2FjdG12YXRv' | base64 -d)
s3=$(awk 'BEGIN{print " "}')
s4=$(cut -c1-9 <<< $(rev <<< 3503ESIRPRUS_))

flag="$s1$s2$s4"
clear > /dev/null
echo "Flag encontrada: $flag"

(cereal@DESKTOP-EM0E755)~/leonardo_Henrique_borges_da_silva-rm566932
$ bash /home/cereal/flag.sh
Flag encontrada: FIAP{wordshell_activato_SURPRISE}

(cereal@DESKTOP-EM0E755)~/leonardo_Henrique_borges_da_silva-rm566932
$
```

A screenshot of a Windows desktop environment. The main focus is a terminal window titled 'cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932'. The terminal shows a shell script being executed, which concatenates several strings (including a hex string, a base64-decoded string, a space, and a reversed string) to reveal a flag: 'FIAP{wordshell_activato_SURPRISE}'. To the right of the terminal, there is a system clock showing '14:04:53' and 'sábado, 30 de maio de 2026', along with a calendar for May 2026 where the 30th is highlighted. The Windows taskbar is visible at the bottom with various icons.

Resultado

A análise do script permitiu identificar diversas técnicas simples de ofuscação utilizadas para ocultar a flag, incluindo codificação hexadecimal, Base64 e manipulação de strings.

Após a correção da concatenação dos fragmentos, foi possível recuperar a flag correta:

FIAP{wordshell_activato_SURPRISE}

Este desafio demonstrou como informações podem ser ocultadas em scripts aparentemente simples por meio da combinação de múltiplas técnicas de codificação e manipulação de texto, exigindo análise detalhada para reconstrução do conteúdo original.

3.15 Hardware Hacking – Ruído Hexadecimal

Enunciado

O desafio consistia na análise de uma captura de comunicação UART proveniente de um dispositivo embarcado conectado a um ambiente industrial.

Segundo o enunciado, durante o processo de inicialização (boot), o equipamento enviava mensagens contendo informações legítimas misturadas com alertas, códigos de diagnóstico e dados corrompidos. Entre essas informações encontrava-se uma flag oculta.

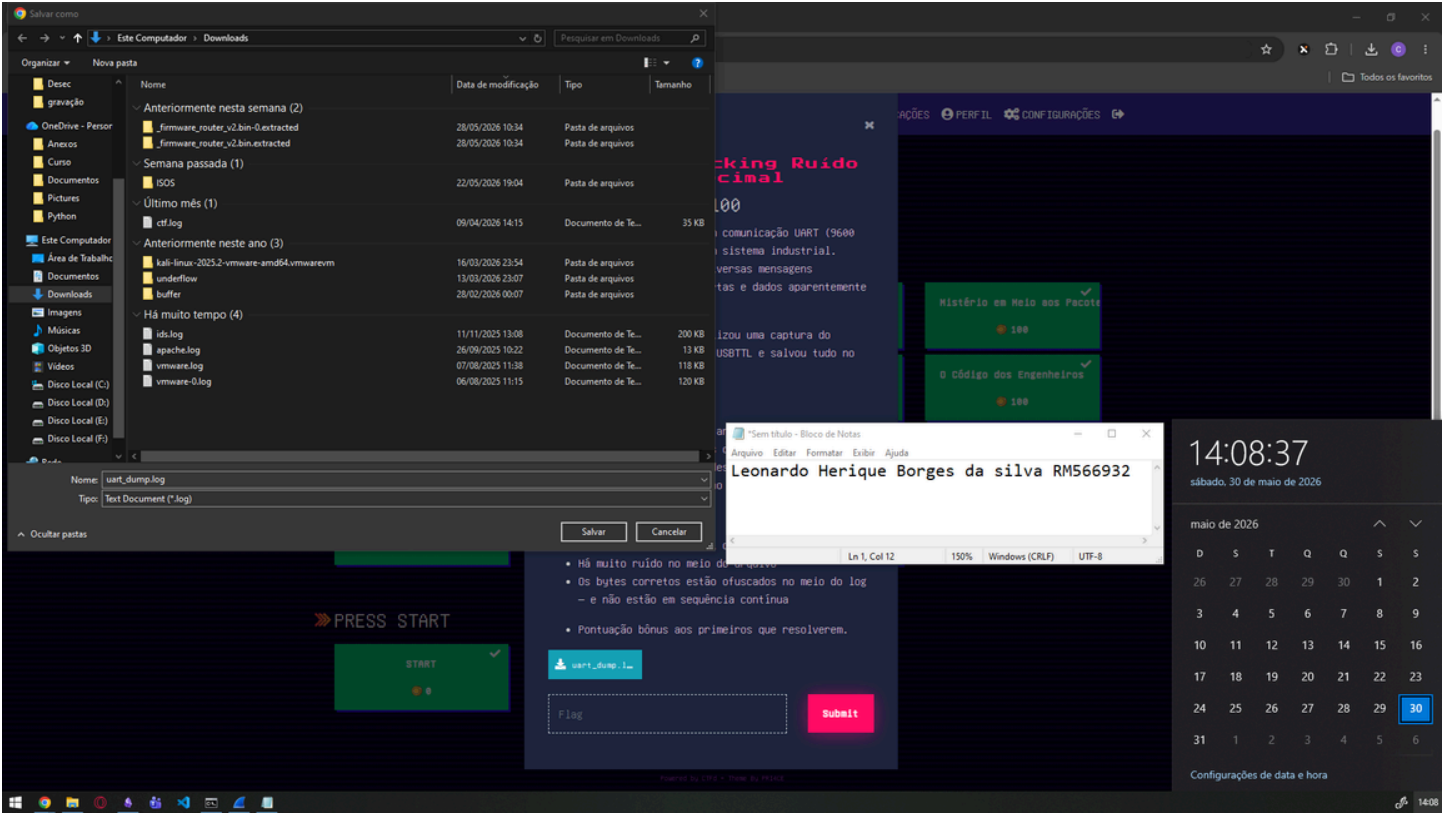
O objetivo era identificar os bytes válidos em meio ao ruído presente no arquivo e reconstruir a string contendo a flag.

Arquivo: uart_dump.log

Metodologia

Inicialmente foi realizado o download do arquivo uart_dump.log disponibilizado pelo desafio.

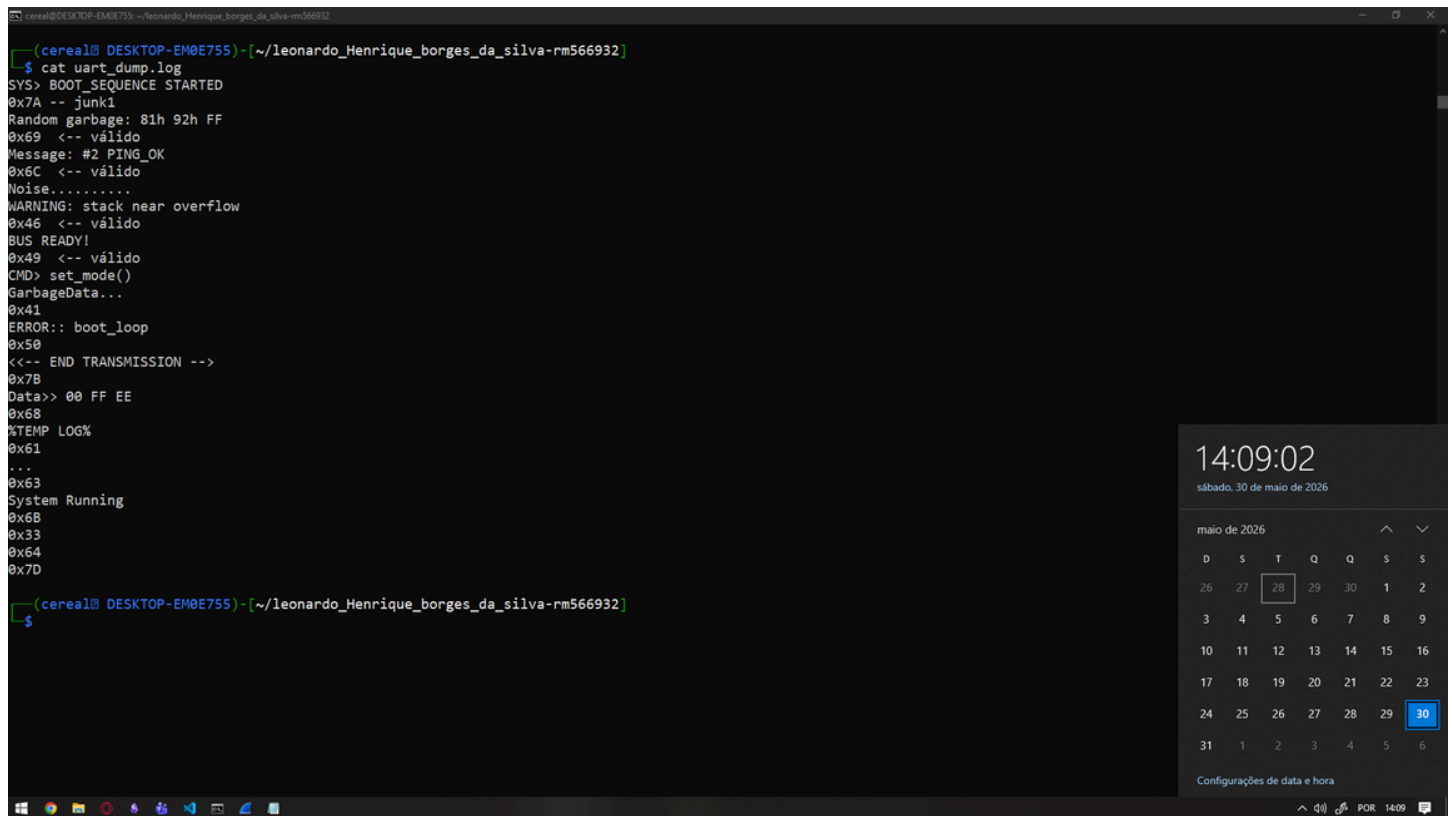
Figura 36 – Arquivo UART disponibilizado para análise



Durante a inspeção manual do conteúdo foi observado que diversas linhas continham a palavra JUNK, indicando dados irrelevantes inseridos para dificultar a análise.

Além disso, os bytes válidos encontravam-se misturados ao ruído, impossibilitando a visualização direta da flag.

Figura 37 – Conteúdo do arquivo UART contendo dados válidos e ruído



The screenshot shows a terminal window with a dark background. The prompt is `(cereal@ DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932`. The user has executed `$ cat uart_dump.log`. The output is a log of UART data, including boot sequence information, random garbage, valid data, and a system running message. The data is presented in hexadecimal format (0xXX). A system clock overlay is visible on the right side of the terminal, showing the time 14:09:02 and the date sábado, 30 de maio de 2026. Below the clock is a calendar for May 2026, with the 30th highlighted. The terminal window has a standard Linux-style taskbar at the bottom with various icons.

```
(cereal@ DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$ cat uart_dump.log
SYS> BOOT_SEQUENCE STARTED
0x7A -- junk1
Random garbage: 81h 92h FF
0x69 <-- válido
Message: #2 PING_OK
0x6C <-- válido
Noise.....
WARNING: stack near overflow
0x46 <-- válido
BUS READY!
0x49 <-- válido
CMD> set_mode()
GarbageData...
0x41
ERROR:: boot_loop
0x50
<<-- END TRANSMISSION -->
0x7B
Data>> 00 FF EE
0x68
XTEMP LOG%
0x61
...
0x63
System Running
0x6B
0x33
0x64
0x7D

(cereal@ DESKTOP-EM0E755) ~/leonardo_Henrique_borges_da_silva-rm566932
$
```

Considerando que os bytes legítimos eram apresentados em formato hexadecimal (0xXX), foi desenvolvido um script em Bash para automatizar o processo de extração.

O script executava as seguintes etapas:

- Filtrava apenas linhas contendo bytes válidos no formato hexadecimal;
- Removia o prefixo 0x;
- Convertia cada byte hexadecimal para seu caractere ASCII correspondente;
- Reconstruía a sequência completa de dados válidos;
- Extraía automaticamente a flag presente na saída.

Figura 38 – Script utilizado para filtragem e reconstrução da flag

```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
(cereal@DESKTOP-EM0E755)~[~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat /home/cereal/hard.sh
#!/bin/bash

LOG_FILE="uart_dump.log"

bytes_hex=$(grep -E "^0x[0-9a-fA-F]{2}" "$LOG_FILE" | sed 's/0x//g' | awk '{print $1}')

dados_brutos=""
for hex in $bytes_hex; do
    caractere=$(echo "$hex" | xxd -r -p)
    dados_brutos="${dados_brutos}${caractere}"
done

flag_extraida=$(echo "$dados_brutos" | grep -o "FIAP{.*}")

echo "Sua flag é: $flag_extraida"

(cereal@DESKTOP-EM0E755)~[~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Após a execução do script, os bytes válidos foram automaticamente convertidos para ASCII, permitindo a reconstrução da mensagem escondida no tráfego UART.

Figura 39 – Execução do script e recuperação da flag

```
cereal@DESKTOP-EM0E755: ~/leonardo_Henrique_borges_da_silva-rm566932
(cereal@DESKTOP-EM0E755)~[~/leonardo_Henrique_borges_da_silva-rm566932]
$ cat /home/cereal/hard.sh
#!/bin/bash

LOG_FILE="uart_dump.log"

bytes_hex=$(grep -E "^0x[0-9a-fA-F]{2}" "$LOG_FILE" | sed 's/0x//g' | awk '{print $1}')

dados_brutos=""
for hex in $bytes_hex; do
    caractere=$(echo "$hex" | xxd -r -p)
    dados_brutos="${dados_brutos}${caractere}"
done

flag_extraida=$(echo "$dados_brutos" | grep -o "FIAP{.*}")

echo "Sua flag é: $flag_extraida"

(cereal@DESKTOP-EM0E755)~[~/leonardo_Henrique_borges_da_silva-rm566932]
$ bash /home/cereal/hard.sh
Sua flag é: FIAP{hack3d}

(cereal@DESKTOP-EM0E755)~[~/leonardo_Henrique_borges_da_silva-rm566932]
$
```

Resultado

A análise do log UART permitiu identificar que apenas parte dos dados presentes na captura eram relevantes, sendo o restante composto por ruído inserido para dificultar a identificação da mensagem.

Utilizando a automação desenvolvida, foi possível extrair os bytes válidos, reconstruir a sequência original e recuperar a flag do desafio.

A flag obtida foi:

FIAP{hack3d}

Com o script desenvolvido, todo o processo de filtragem, conversão hexadecimal, reconstrução da sequência e extração da flag foi realizado automaticamente em menos de um segundo, garantindo rapidez, precisão e reprodutibilidade da análise.

Este desafio demonstrou conceitos relacionados à análise de comunicações UART, interpretação de dados em hexadecimal e automação de tarefas forenses para recuperação de informações ocultas em dispositivos embarcados.

4 CONCLUSÃO

O desenvolvimento dos desafios propostos permitiu a aplicação prática de diversos conceitos abordados ao longo do curso de Defesa Cibernética, abrangendo áreas como criptografia, análise forense, engenharia reversa, análise de tráfego de rede, segurança em redes sem fio, sistemas embarcados e automação de processos de investigação.

Durante a execução dos challenges foi possível empregar diferentes metodologias de análise, utilizando ferramentas amplamente adotadas no mercado de segurança da informação, como Wireshark, Tshark, Binwalk, John the Ripper e scripts desenvolvidos em Python e Bash. A utilização dessas ferramentas possibilitou a identificação de padrões ocultos, extração de informações relevantes, reconstrução de dados fragmentados e recuperação de evidências digitais presentes em diferentes cenários.

Os desafios também contribuíram para o desenvolvimento do raciocínio analítico e investigativo, exigindo a interpretação de artefatos digitais, identificação de técnicas de ofuscação, análise de protocolos de rede e compreensão do funcionamento de dispositivos embarcados. Em diversas situações, foi necessária a criação de scripts para automatizar tarefas de análise, reduzindo o tempo de investigação e aumentando a precisão dos resultados obtidos.

Além dos aspectos técnicos, a atividade proporcionou uma experiência próxima de situações encontradas em ambientes reais de segurança ofensiva, resposta a incidentes e perícia computacional, permitindo consolidar conhecimentos teóricos por meio da resolução prática de problemas.

Dessa forma, o conjunto de desafios propostos atingiu seu objetivo educacional ao estimular a aplicação integrada de técnicas e ferramentas de cibersegurança, fortalecendo competências fundamentais para atuação profissional na área de Defesa Cibernética.

5 REFERÊNCIAS

BINWALK. Firmware Analysis Tool. Disponível em: <https://github.com/ReFirmLabs/binwalk>. Acesso em: 30 maio 2026.

JOHN THE RIPPER. Password Cracking and Auditing Tool. Disponível em: <https://www.openwall.com/john/>. Acesso em: 30 maio 2026.

PYTHON SOFTWARE FOUNDATION. Python Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 30 maio 2026.

WIRESHARK FOUNDATION. Wireshark User Guide. Disponível em: <https://www.wireshark.org/docs/>. Acesso em: 30 maio 2026.

WIRESHARK FOUNDATION. Tshark Manual Page. Disponível em: <https://www.wireshark.org/docs/man-pages/tshark.html>. Acesso em: 30 maio 2026.

SITE112. Cifra de César – Codificar e Descodificar. Disponível em: <https://site112.com/cifra-de-cesar-codificar-descodificar>. Acesso em: 30 maio 2026.

X64DBG DEVELOPMENT TEAM. x64dbg Documentation. Disponível em: <https://x64dbg.com/>. Acesso em: 30 maio 2026.